



# RAG Pipeline

*AICB-P1 · Ngày 8 · Truy Xuất & Sinh Câu  
Trả Lời*

---

**Tên Giảng Viên**

VinUniversity · Phase 1 · Tuần 2 ·  
2026

## HÃY SUY NGHĨ...

*"Bạn đã build agent với vector store. Nhưng agent vẫn hallucinate và trả lời sai. Lỗi nằm ở đâu?"*



Giữ câu hỏi này trong đầu khi học bài hôm nay

1. The RAG Paradigm & Indexing Architecture
2. Query Processing & Advanced Retrieval
3. Generation, Grounding & UX
4. The RAG Evaluation Triad
5. Lab 8 + deliverable

- Giải thích được **RAG** như một pipeline gồm indexing, retrieval, re-ranking, và generation
- Hiểu vì sao **retrieval quality** thường quyết định chất lượng câu trả lời nhiều hơn prompt viết đẹp
- So sánh được **dense, sparse, hybrid retrieval** và biết khi nào cần rerank
- Thiết kế được **prompt grounding** để model trả lời từ context thay vì bịa thêm
- Đo được chất lượng RAG bằng **faithfulness, relevance, context recall/completeness**



Full RAG pipeline với index, retrieval, answer function, 10 test questions, và scorecard đánh giá chất lượng

- 1 pipeline **index** → **retrieve** → **rerank/select** → **generate**
- 1 bộ câu hỏi test có **expected evidence** hoặc expected answer
- 1 bảng điểm ngắn để so sánh baseline và bản tuning đầu tiên

# The RAG Paradigm & Indexing Architecture

---

RAG is not just adding context; it is a synergistic orchestration of indexing, retrieval, and generation systems to ensure factual grounding and accuracy.



## **1.1 The Need for RAG**

---

Understanding why standard LLMs fall short and how RAG bridging the gap between static knowledge and dynamic, factual accuracy.

## Kiến thức bị đóng băng (Knowledge Cutoff)

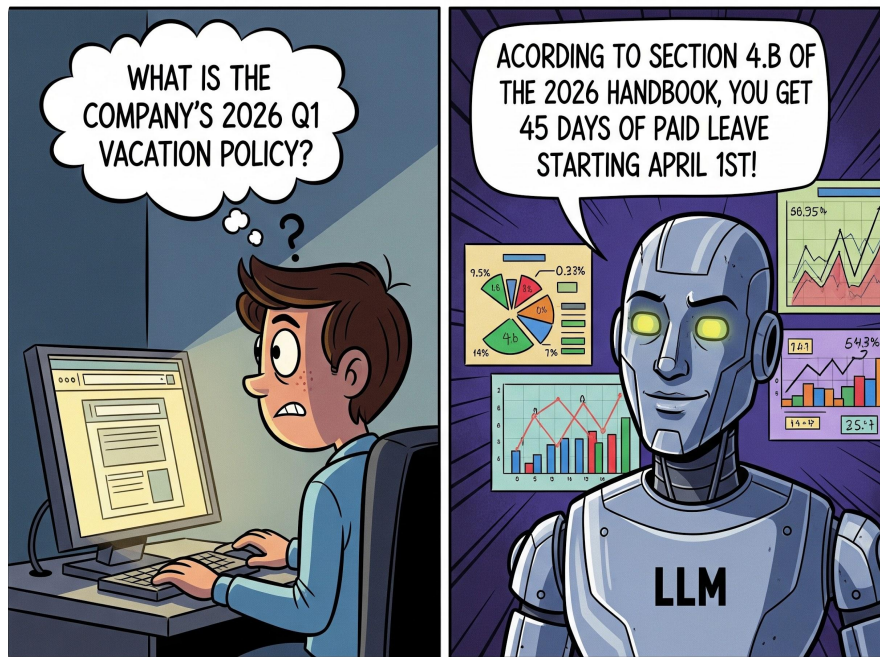
LLM chỉ biết những gì đã xảy ra trước ngày training. Thông tin nội bộ hay sự kiện mới là điểm mù.

## Bản chất xác suất (Probabilistic Nature)

LLM là cỗ máy dự đoán từ tiếp theo, ưu tiên sự trôi chảy (fluency) hơn tính chính xác (factual accuracy).

## Hệ quả - Hallucination

Khi thiếu dữ kiện, model sẽ tự động "bịa" ra thông tin trông rất logic và tự tin để làm hài lòng người dùng.



# Fine-tuning vs. RAG (Hai Cách Tiếp Cận Khác Nhau)

## Fine-tuning (Học phong cách)

Phù hợp để thay đổi cách model nói chuyện (tone, format), nhưng cực kỳ kém và đắt đỏ nếu dùng để nhồi nhét sự kiện (facts). Trí nhớ mạng nơ-ron rất dễ bị "catastrophic forgetting".

## RAG (Cung cấp tài liệu)

Phù hợp để truy xuất thông tin thực tế. Giống như việc cho học sinh mang tài liệu vào phòng thi "Open-book exam" thay vì bắt học thuộc lòng.

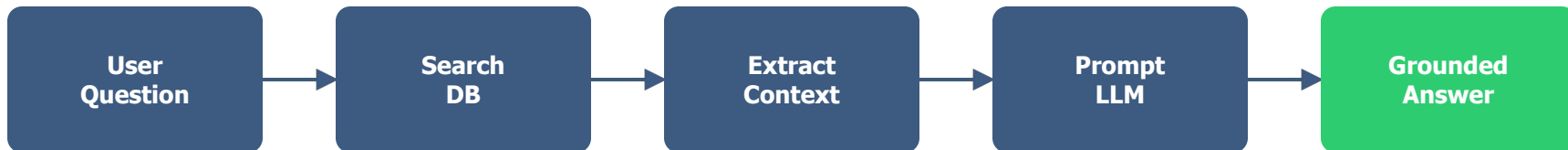
| Metric                 | Fine-tuning               | RAG                             |
|------------------------|---------------------------|---------------------------------|
| Cost to Update         | Cao (Retraining required) | Thấp (Update index)             |
| Risk of Hallucination  | Cao (Static knowledge)    | Thấp (Grounded in facts)        |
| Dynamic Access Control | Khó (All-in-one weights)  | Dễ (Document-level permissions) |

## Sự kết hợp của 2 cỗ máy:

RAG ghép nối sức mạnh tìm kiếm của một Search Engine với khả năng tổng hợp ngôn ngữ của LLM.

## Quy trình ngược:

Thay vì hỏi model ngay lập tức, ta chặn câu hỏi lại → dùng nó để truy vấn cơ sở dữ liệu → lấy bài viết liên quan nhất → ép model đọc bài viết đó để trả lời.





## **Nguồn gốc rõ ràng**

(Auditability): Mọi câu trả lời đều có thể đính kèm đường link trích dẫn (citation). Nếu AI trả lời sai, ta biết ngay là do tài liệu sai hay do AI suy diễn.



## **Bảo mật & Phân quyền**

(Access Control / RBAC): LLM không lưu dữ liệu. Người dùng A ở phòng Marketing chỉ được phép search (retrieve) các tài liệu mà họ có quyền xem.



## **Cập nhật Real-time**

(Freshness): Khi chính sách thay đổi, chỉ cần xóa file cũ / thêm file mới vào Vector DB. Không cần train lại model.



Các model (GPT-4, Gemini, Claude) đang dần trở thành **"hàng hóa cơ bản" (commodity)** với sức mạnh tương đương nhau.



Sự khác biệt của một sản phẩm AI doanh nghiệp nằm ở hệ thống dữ liệu. **Pipeline xử lý, làm sạch và tìm kiếm dữ liệu** mới là lợi thế cạnh tranh cốt lõi.



**"Rác vào, rác ra" (Garbage In, Garbage Out):** Nếu retrieval mang về thông tin nhiễu, prompt kỹ thuật đến đâu cũng vô dụng.



**Your LLM is only as smart as your retrieval system.**

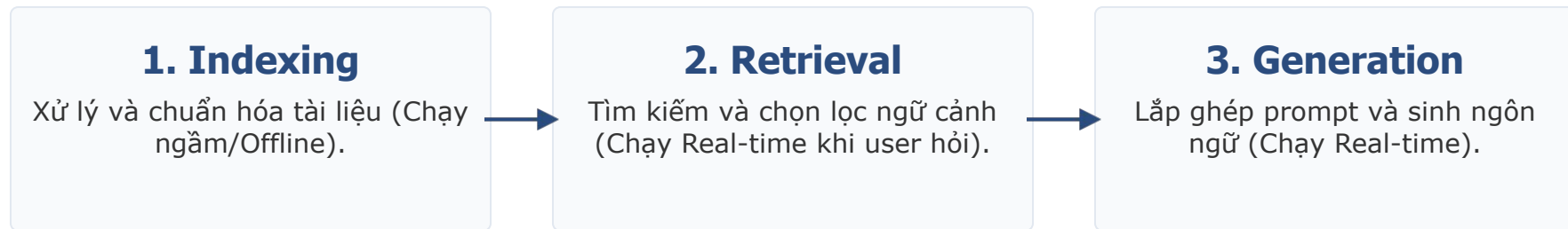


## **1.2 High-level RAG Architecture**

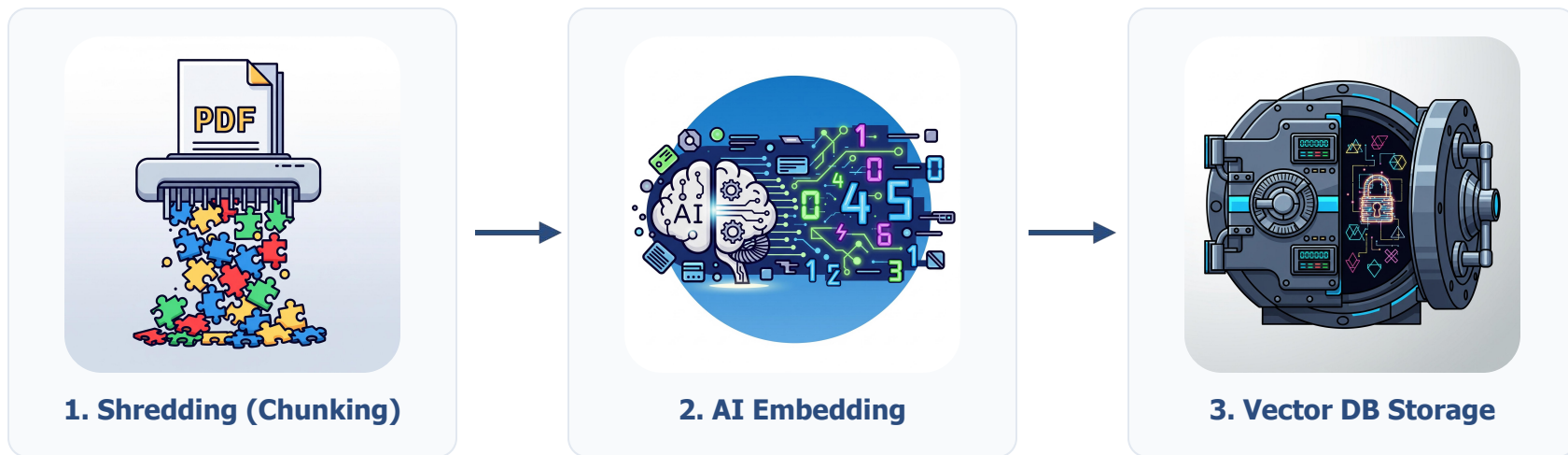
---

The anatomy of a RAG pipeline: How Indexing, Retrieval, and Generation work together to build a reliable search-and-synthesize engine.

RAG không phải là một hàm API gọi một lần. Nó là một hệ thống phân tán gồm 3 khối kiến trúc riêng biệt chạy nối tiếp nhau:



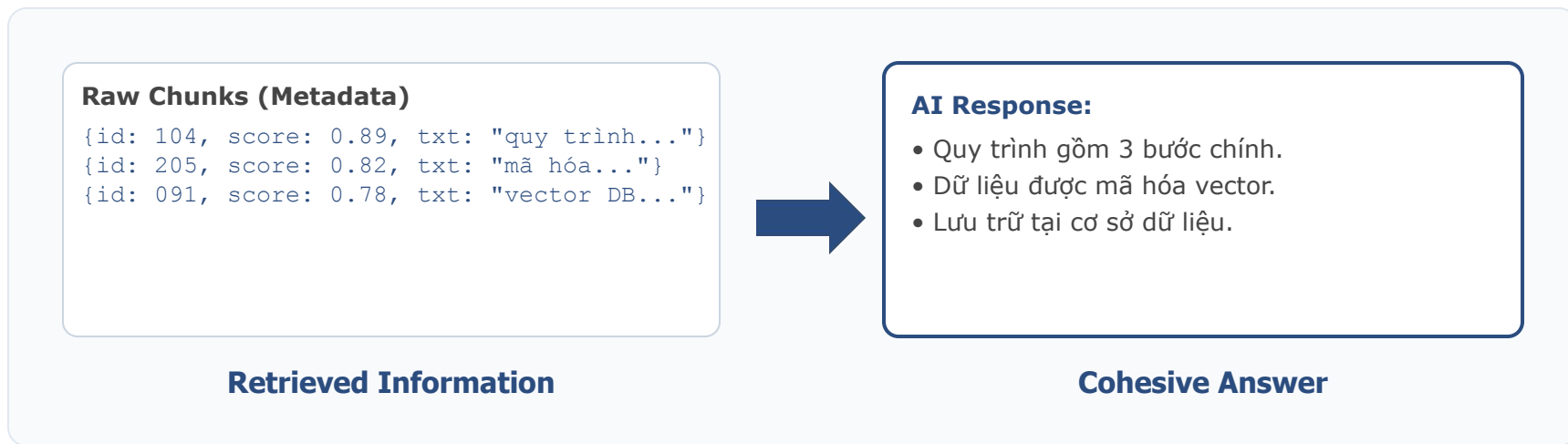
- Đây là quá trình ETL (Extract, Transform, Load) dành cho dữ liệu phi cấu trúc.
- Mục tiêu: Biến các file PDF, Word, HTML khổng lồ thành các đoạn thông tin nhỏ (chunks), mã hóa chúng thành số (vectors), và lưu vào Database chuyên dụng.



- Khi user đặt câu hỏi, hệ thống cũng phải mã hóa câu hỏi đó thành vector bằng đúng model đã dùng ở bước Indexing.
- Sử dụng thuật toán k-NN (K-Nearest Neighbors) hoặc ANN (Approximate Nearest Neighbors) để tính khoảng cách trong không gian toán học, từ đó rút ra top K đoạn văn bản gần nghĩa nhất.

# Bước 3 - Generation Pipeline (Tổng Hợp & Trình Bày)

- Thông tin thô từ DB rất lộn xộn và khó đọc. LLM đóng vai trò là "biên tập viên".
- Đưa toàn bộ ngữ cảnh tìm được vào System Prompt kèm theo lệnh giới hạn nghiêm ngặt: "Chỉ trả lời dựa trên tài liệu được cung cấp".
- Xử lý ngoại lệ: Nếu DB trả về kết quả rỗng, LLM phải được lập trình để xin lỗi và báo thiếu dữ liệu.



Khi test RAG thấy kết quả sai, kỹ sư thường vội vàng nhảy vào sửa Prompt hoặc đổi model lớn hơn. Đây là sai lầm!

**80%**

## Lỗi do Retrieval

- Truy vấn tìm sai tài liệu
- Thiếu chứng cứ quan trọng
- Nhồi quá nhiều rác (noise)

**20%**

## Lỗi do Generation

- Model bỏ qua chứng cứ
- Ảo giác sinh thêm chi tiết
- Định dạng sai

## **1.3 Document Parsing & Ingestion**

---

Tackle the complexities of parsing multi-column PDFs, extracting nested tables, and building a robust ingestion pipeline to feed your database.

Các khóa học thường demo bằng file .txt sạch sẽ. Thực tế doanh nghiệp là:

- PDF scan (hóa đơn, hợp đồng)
- Email cũ (nhiều ký tự lạ)
- Slide thuyết trình (layout phức tạp)

## Thách thức OCR

OCR kém sẽ đọc chữ "I" thành số "1", làm hỏng toàn bộ keyword quan trọng.

| layer name | output size | 18-layer                         | 34-layer            | 50-layer            | 101-layer           | 152-layer            |
|------------|-------------|----------------------------------|---------------------|---------------------|---------------------|----------------------|
| conv1      | 112×112     | 7×7, 64, stride 2                |                     |                     |                     |                      |
| conv2_x    | 56×56       | 3×3 max pool, stride 2           |                     |                     |                     |                      |
|            |             | 7×7, 64, stride 2                |                     |                     |                     |                      |
|            |             | 3×3 max pool, stride 2           |                     |                     |                     |                      |
| conv3_x    | 28×28       | 3×3, 128                         |                     |                     |                     |                      |
|            |             | 3×3, 128                         |                     |                     |                     |                      |
| conv4_x    | 14×14       | 3×3, 256                         |                     |                     |                     |                      |
|            |             | 3×3, 256                         |                     |                     |                     |                      |
| conv5_x    | 7×7         | 3×3, 512                         |                     |                     |                     |                      |
|            |             | 3×3, 512                         |                     |                     |                     |                      |
|            | 1×1         | average pool, 1000-d fc, softmax |                     |                     |                     |                      |
| FLOPs      |             | 1.8×10 <sup>9</sup>              | 3.6×10 <sup>9</sup> | 3.8×10 <sup>9</sup> | 7.8×10 <sup>9</sup> | 11.3×10 <sup>9</sup> |

Table 1. Architectures for ImageNet. Building blocks are shown in brackets (see also Fig. 5), with the numbers of blocks stacked. Down-sampling is performed by conv3\_1, conv4\_1, and conv5\_1 with a stride of 2.

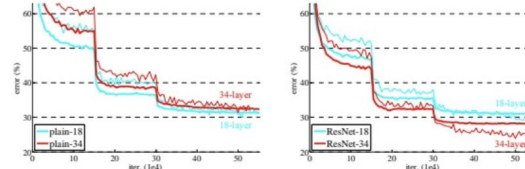


Figure 4. Training on ImageNet. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

|           | plain | ResNet       |
|-----------|-------|--------------|
| 18 layers | 27.94 | 27.88        |
| 34 layers | 28.54 | <b>25.03</b> |

Table 2. Top-1 error (%; 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

34-layer plain net has higher *training* error throughout the whole training procedure, even though the solution space of the 18-layer plain network is a subspace of that of the 34-layer one.

We argue that this optimization difficulty is *unlikely* to be caused by vanishing gradients. These plain networks are trained with BN [16], which ensures forward propagated signals to have non-zero variances. We also verify that the backward propagated gradients exhibit healthy norms with BN. So neither forward nor backward signals vanish. In fact, the 34-layer plain net is still able to achieve competitive accuracy (Table 3), suggesting that the solver works to some extent. We conjecture that the deep plain nets may have exponentially low convergence rates, which impact the

reducing of the training error<sup>3</sup>. The reason for such optimization difficulties will be studied in the future.

**Residual Networks.** Next we evaluate 18-layer and 34-layer residual nets (*ResNets*). The baseline architectures are the same as the above plain nets, except that a shortcut connection is added to each pair of 3×3 filters as in Fig. 3 (right). In the first comparison (Table 2 and Fig. 4 right), we use identity mapping for all shortcuts and zero-padding for increasing dimensions (option A). So they have *no extra parameter* compared to the plain counterparts.

We have three major observations from Table 2 and Fig. 4. First, the situation is reversed with residual learning – the 34-layer ResNet is better than the 18-layer ResNet (by 2.8%). More importantly, the 34-layer ResNet exhibits considerably lower training error and is generalizable to the validation data. This indicates that the degradation problem is well addressed in this setting and we manage to obtain accuracy gains from increased depth.

Second, compared to its plain counterpart, the 34-layer

<sup>3</sup>We have experimented with more training iterations (3×) and still observed the degradation problem, suggesting that this problem cannot be feasibly addressed by simply using more iterations.



## Vấn Đề Kỹ Thuật

- Chuẩn PDF sinh ra để in ấn, nó lưu tọa độ (x, y) của chữ chứ không hiểu cấu trúc ngữ nghĩa (đâu là tiêu đề, đâu là đoạn văn).
- **Lỗi layout:** Các parser cơ bản thường đọc từ trái sang phải, làm trộn lẫn văn bản giữa 2 cột riêng biệt thành một câu vô nghĩa.
- **Nhiều Header/Footer:** Số trang và tiêu đề lặp lại ở mọi trang sẽ làm bẩn database nếu không được gỡ bỏ.

Nếu dùng thuật toán cắt text thông thường, bảng bị xẻ làm đôi. Nửa dưới mất liên kết với Header → **Vector vô dụng.**

**Giải pháp tối ưu:** Phải dùng parser chuyên dụng (LlamaParse, Unstructured) để bóc tách thành HTML/Markdown.

## Raw Text "Soup"

Basic Pro Enterprise  
\$9/month \$29/month \$99/month  
10Gb Storage 50GB Storage  
Unlimited Storage

## Original PDF Table

| Basic         | Pro              | Enterprise                |
|---------------|------------------|---------------------------|
| \$9/month     | \$29/month       | \$99/month                |
| 10GB Storage  | 50GB Storage     | Unlimited Storage         |
| 5 Users       | 25 Users         | Unlimited Users           |
| Users         | Priority Support | 24/7 Phone Support        |
| Email Support | Priority Support | SLA                       |
|               | Custom Branding  | Dedicated Account Manager |
| Choose Basic  | Choose Pro       | Contact Us                |

## Markdown/HTML

```
| Basic | Pro | Enterprise |  
|$9/month | $29/month | $99/month |  
|---|---|---|  
|10Gb Storage | 50GB Storage |  
Unlimited Storage |
```

Thay vì dùng công cụ bóc chữ truyền thống, ta ném thẳng ảnh chụp trang tài liệu cho Vision LLMs (như Gemini 1.5 Pro hoặc GPT-4o).

## Ưu điểm:

- Hiểu được cấu trúc siêu phức tạp.
- Đọc được biểu đồ (charts).
- Lưu lại được diễn giải hình ảnh.

## Nhược điểm:

- Tốn kém chi phí API.
- Chạy chậm (không phù hợp cho kho dữ liệu hàng triệu trang).

## Visual Pipeline

### Input Prompt:

"Analyze this flowchart image and return a JSON object describing the nodes and edges. Structure it with 'nodes' (id, text, type) and 'edges' (from\_node\_id, to\_node\_id, label)." 

### Sample JSON Output:

```
json
{
  "flowchart": {
    "nodes": [
      { "id": "1", "text": "Start", "type": "start" },
      { "id": "2", "text": "User Login", "type": "process" },
      { "id": "3", "text": "Valid?", "type": "decision" },
      { "id": "4", "text": "Error Message", "type": "process" },
      { "id": "5", "text": "Dashboard", "type": "process" }
    ],
    "edges": [
      { "from": "1", "to": "2", "label": "" },
      { "from": "2", "to": "3", "label": "" },
      { "from": "3", "to": "4", "label": "No" },
      { "from": "3", "to": "5", "label": "Yes" }
    ]
  }
}
```

**Chuẩn hóa (Normalization):** Gộp các khoảng trắng thừa, sửa lỗi unicode (font tiếng Việt cũ), xóa các ký tự điều khiển (control characters) làm rối model.

**Redaction (Che mờ PII):** Xóa thông tin cá nhân (CCCD, số thẻ tín dụng, số điện thoại) trước khi đưa lên Cloud Vector DB để đảm bảo tuân thủ bảo mật (GDPR/PDPA).

## Ví dụ minh họa (Example)

```
"KH Nguyễn Văn A, \n\n SDT:  
0901234567"
```



```
Cleaned: "KH [REDACTED], SDT:  
[REDACTED]"
```

## Batch Processing

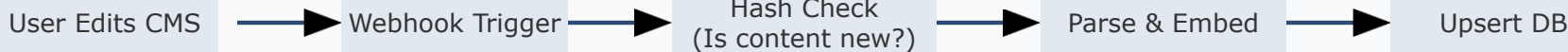
Cập nhật định kỳ (VD: quét lại toàn bộ Google Drive vào 12h đêm). Dễ triển khai nhưng thông tin bị trễ (stale data).

## Event-Driven (Delta Sync)

Dùng Webhook bắt sự kiện. Chỉ khi nào có user sửa file trên Confluence, hệ thống mới trigger job cập nhật riêng file đó.

**Idempotency:** Phải thiết kế hệ thống băm (hashing) nội dung để tránh việc lưu trùng lặp một chunk văn bản nhiều lần.

## Visual Workflow: Event-Driven Pipeline



```
graph LR; A[User Edits CMS] --> B[Webhook Trigger]; B --> C[Hash Check  
(Is content new?)]; C --> D[Parse & Embed]; D --> E[Upsert DB];
```

The diagram illustrates an event-driven pipeline for data ingestion. It starts with 'User Edits CMS', which triggers a 'Webhook Trigger'. This leads to a 'Hash Check (Is content new?)' step, which then proceeds to 'Parse & Embed', and finally to 'Upsert DB'.

## **1.4 Advanced Chunking Strategies**

---

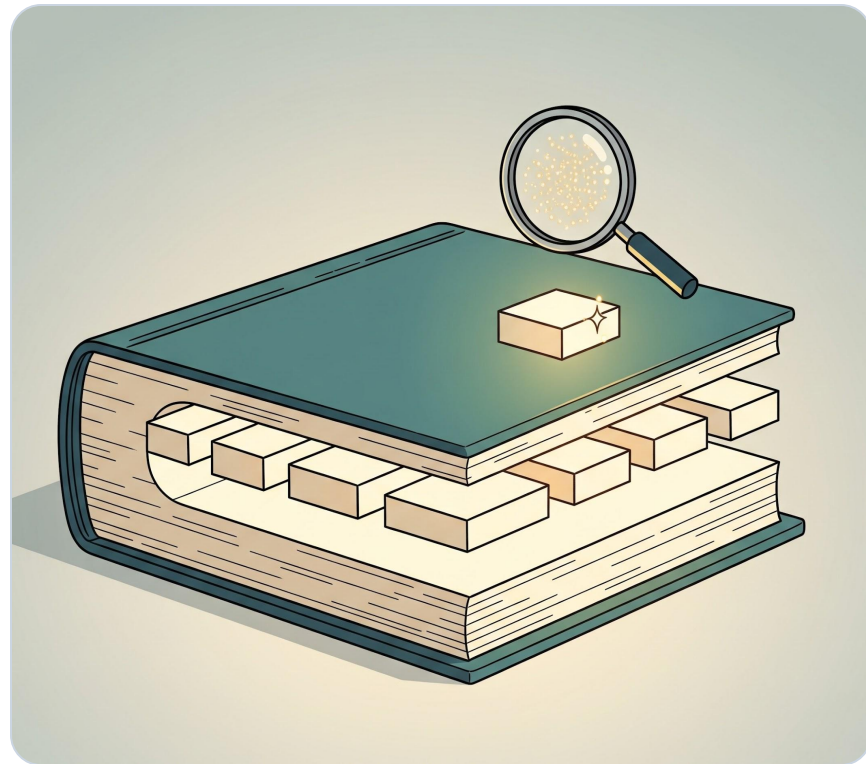
Beyond naive character splitting: Exploring recursive, structural, and semantic chunking strategies to preserve document context and maximize retrieval accuracy.

## Giới hạn Context Window

Dù LLM hiện nay hỗ trợ ngữ cảnh dài (ví dụ Gemini 1.5 hỗ trợ 2M tokens), việc nhét cả nghìn trang tài liệu vào prompt rất đắt tiền và tốn thời gian phản hồi (TTFT).

## Giảm nhiễu (Noise)

Vector Search tìm "mật độ" ý nghĩa. Nếu nhúng cả một chương sách vào 1 vector, các ý chính sẽ bị pha loãng, rất khó khớp với câu hỏi cụ thể của user.



## Đặc điểm & Đánh giá

- Cắt cơ học theo số lượng ký tự hoặc token (Ví dụ: Cứ 500 ký tự thì chém 1 nhát).

### Ưu điểm:

Cực kỳ dễ code, chạy nhanh, dự đoán được chính xác dung lượng database.

### Nhược điểm (Tử huyệt):

Rất dễ cắt ngang một câu, làm đứt đoạn ngữ nghĩa (ví dụ: cắt giữa chữ "không" và "được phép").

Chính sách này áp dụng cho toàn bộ nhân viên ngoại | trừ thực tập sinh.

*Nghĩa của câu bị chia đôi tại điểm cắt*



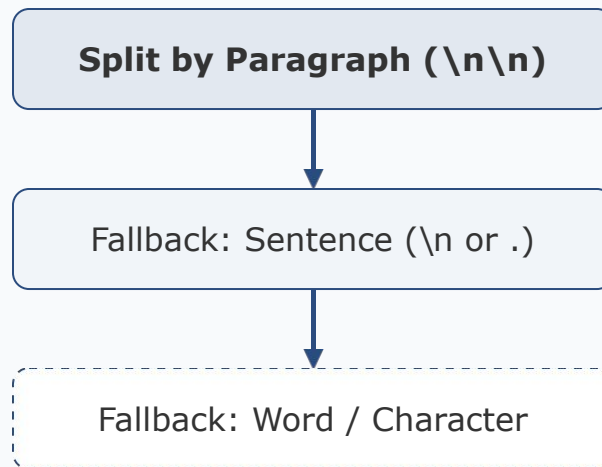
## Đặc điểm & Cách hoạt động

- Đây là tiêu chuẩn vàng mặc định (Default standard) trong LangChain.

Thay vì cắt mù quáng, nó cố gắng tôn trọng ranh giới ngôn ngữ bằng cách thử cắt theo thứ tự ưu tiên:

1. Cắt ở khoảng trống giữa 2 đoạn văn (`\n\n`).
2. Nếu đoạn văn quá dài, cắt ở ký tự xuống dòng (`\n`).
3. Nếu vẫn dài, cắt ở dấu chấm câu (`.`).

## Fallback Logic Tree



## Đặc điểm & Ứng dụng

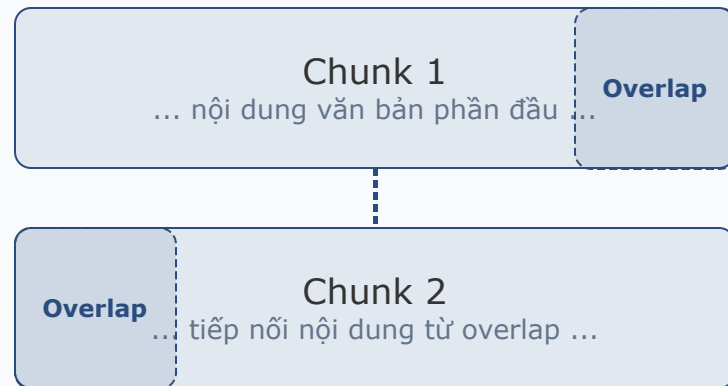
- Không quan tâm độ dài, chỉ cắt dựa trên cấu trúc logic của tài liệu.
- **Markdown/HTML:** Tách riêng các phần dưới thẻ <H1>, <H2>.
- **Code:** Dùng `AST (Abstract Syntax Tree)` để tách riêng từng function (hàm) hoặc class, đảm bảo không một hàm Python nào bị cắt đứt làm đôi.

# Heading 1

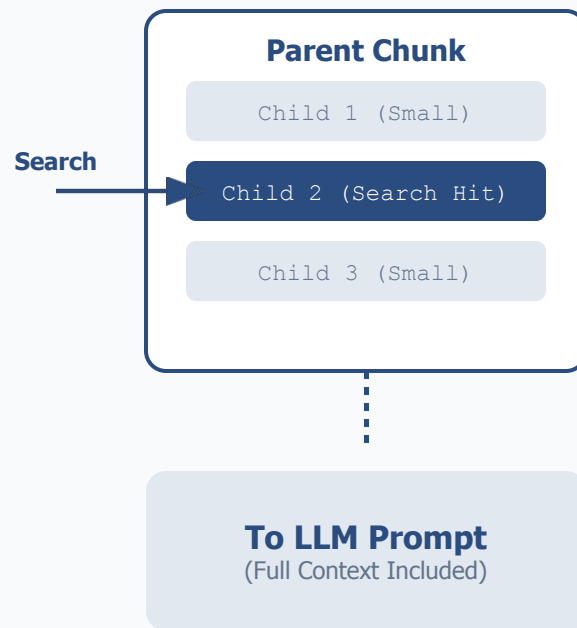
## Sub-heading A

## Sub-heading B

- Khi chia văn bản, một ý quan trọng có thể vô tình bị chia làm 2 mảnh nằm ở mép của 2 chunk khác nhau.
- **Overlap (Khoảng lặp lại):** Cho phép đoạn cuối của Chunk 1 được lặp lại ở đoạn đầu của Chunk 2 (thường set khoảng 10-15% tổng size).
- Việc này hoạt động như chất "keo dính" giữ lại mạch ngữ cảnh.



- **Vấn đề:** Chunk nhỏ thì search chính xác nhưng thiếu ngữ cảnh. Chunk to thì search dễ trượt nhưng ngữ cảnh dồi dào.
- **Giải pháp Parent-Child:** Lưu trữ các chunk là những câu siêu ngắn (Small) để chạy Vector Search lấy độ chính xác cao.
- Khi tìm trúng câu nhỏ đó, hệ thống sẽ tự động móc nối và gửi toàn bộ đoạn văn lớn chứa câu đó (Parent) vào LLM Prompt.



- Bảng biểu không có câu văn hoàn chỉnh, vector search rất khó bắt nghĩa.
- **Cách 1 - Row to Text:** Biến từng dòng của bảng thành một câu văn tự nhiên (VD: "Sản phẩm iPhone 15 có giá 20 triệu, tồn kho 5 chiếc").
- **Cách 2 - LLM Summarization:** Dùng LLM đọc toàn bộ bảng, viết 1 đoạn tóm tắt ý chính của bảng đó, và lưu đoạn tóm tắt đó thành vector đại diện cho cái bảng.



## Chunking tệ

- cắt giữa một bảng hoặc điều khoản
- quá to: nhiều ý không liên quan
- quá nhỏ: mất ngữ cảnh và thiếu source

### Ví dụ:

Điều kiện hoàn tiền được áp dụng khi...  
... khách hàng gửi yêu cầu trong vòng 7 ngày làm việc kể từ thời điểm xác nhận...

## Chunking tốt

- cắt theo heading, section, paragraph tự nhiên
- có overlap vừa đủ
- giữ source, section, date
- retriever hiểu ngữ nghĩa trọn vẹn hơn

### Ví dụ:

#### **Hoàn tiền - Điều kiện áp dụng**

Yêu cầu được gửi trong 7 ngày...

**Source:** policy/refund-v4.pdf · Điều 3

## 1.5 Embeddings & Metadata

---

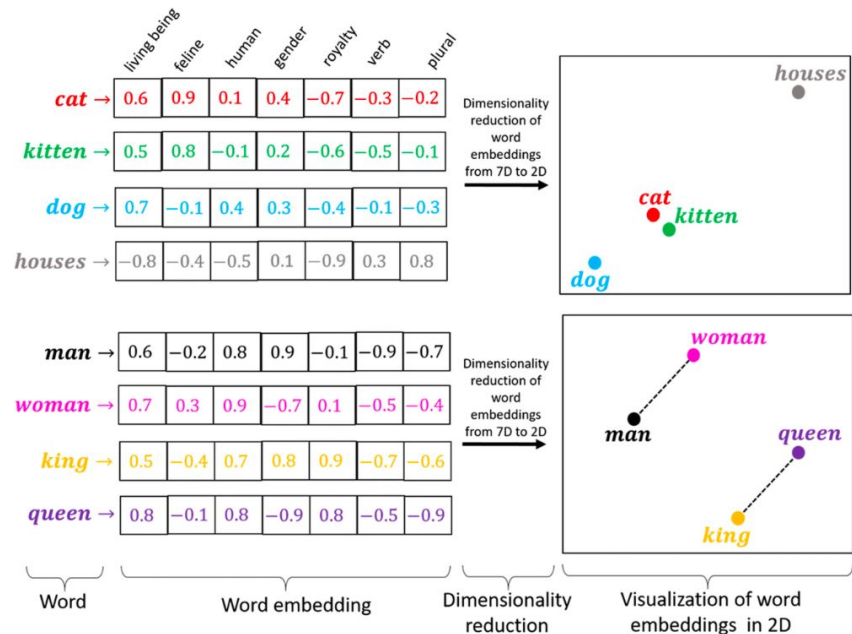
Beyond naive character splitting: Exploring recursive, structural, and semantic chunking strategies to preserve document context and maximize retrieval accuracy.

## Bản chất của Embedding

Mô hình máy học không hiểu được ngôn ngữ của con người. Embedding là phép biến đổi toán học chuyển một câu thành một dải số (Vector).

## Không gian ngữ nghĩa

Các câu có ý nghĩa giống nhau (dù dùng từ vựng hoàn toàn khác nhau) sẽ có điểm tọa độ nằm sát nhau trong không gian đa chiều (Semantic space).





## Số chiều (Dimensions)

Model nhỏ (như bge-micro, 384 chiều) tính toán siêu nhanh, ít tốn RAM.

Model lớn (OpenAI, 1536 chiều) phân biệt nghĩa tinh tế hơn nhưng chi phí hạ tầng cao gấp 4 lần.

## Rào cản ngôn ngữ

Đừng dùng model chuyên tiếng Anh cho văn bản tiếng Việt.

Phải chọn các model Multilingual (m-E5, Cohere Multilingual) để mapping ngôn ngữ tốt.

MTEB(Multilingual, v2)

The large-scale multilingual expansion of MTEB known as MMTEB, driven mainly by highly-curated community contributions covering 250+ languages.

◦ Number of languages: 1038

◦ Number of tasks: 131

◦ Number of task types: 9

◦ Number of domains: 20

Cite and share this benchmark

Customize this Benchmark

Advanced Model Filters

[Click for More Info](#)

Summary

Performance per Model Size

Performance over Time

Performance per Task Type

Performance per task

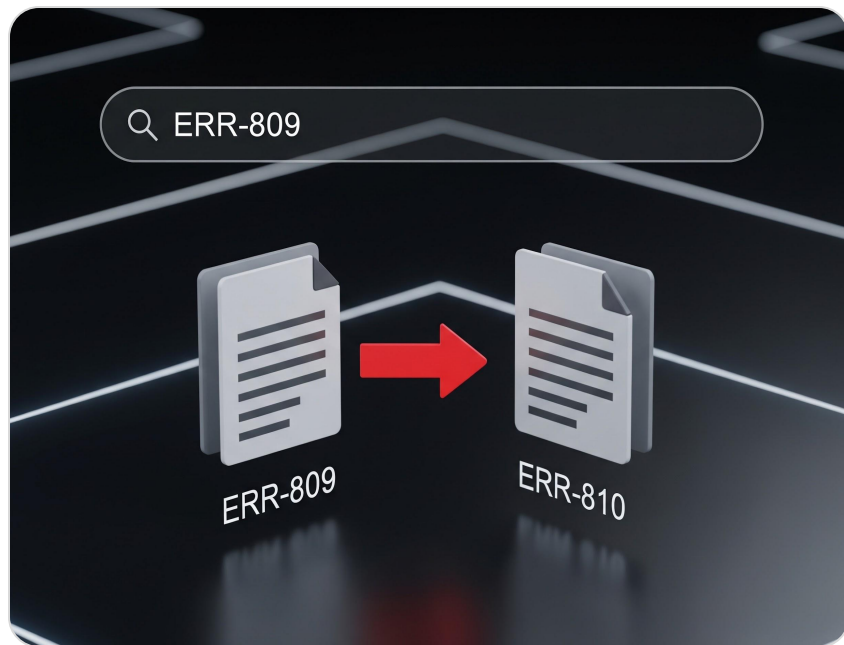
...

Filter...

| Rank (Bo... | Model                                           | nstruction R... | Multilabel Class... | Pair Classificat... | Reranking | Retrieval | STS   |
|-------------|-------------------------------------------------|-----------------|---------------------|---------------------|-----------|-----------|-------|
| 1           | <a href="#">harrier-oss-v1-27b</a>              | .39             | 36.44               | 85.44               | 67.35     | 78.27     | 79.99 |
| 2           | <a href="#">KallM-Embedding-Gemma3-12B-2511</a> | .49             | 33.03               | 84.73               | 67.27     | 75.66     | 79.02 |
| 3           | <a href="#">llama-embed-nemotron-8b</a>         | 0.82            | 29.86               | 83.97               | 67.78     | 68.69     | 79.41 |
| 4           | <a href="#">Qwen3-Embedding-8B</a>              | 0.06            | 28.66               | 86.40               | 65.63     | 70.88     | 81.08 |
| 5           | <a href="#">gemini-embedding-001</a>            | .18             | 29.16               | 83.63               | 65.58     | 67.71     | 79.40 |
| 6           | <a href="#">Qwen3-Embedding-4B</a>              | 1.56            | 26.77               | 85.05               | 65.08     | 69.60     | 80.86 |
| 7           | <a href="#">Octen-Embedding-8B</a>              | .90             | 25.23               | 85.12               | 67.64     | 71.61     | 81.27 |

## Hạn chế của Vector Search

- Vector hoàn hảo trong việc hiểu "ý nghĩa" và "diễn đạt lại".
- Tuy nhiên, Vector cực kỳ tồi tệ khi đối mặt với **Exact Match** (Khớp chính xác): Mã hợp đồng, ID lỗi (ERR-809), số series.
- Model nhúng có thể xếp ERR-809 và ERR-810 sát cạnh nhau vì cấu trúc giống nhau, dẫn đến trả lời nhầm.



## Tại sao Metadata lại quan trọng?

- Đừng bao giờ chỉ đẩy "Raw Text" vào Database. Việc gắn thẻ dữ liệu (Tagging/Metadata) là bắt buộc.
- Một chunk tốt phải mang theo "giấy tờ tùy thân": source\_file, doc\_type, date\_created, department\_owner.
- Lợi ích: Cho phép cắt giảm không gian tìm kiếm trước khi chạy thuật toán vector nặng nề.

```
// JSON Object Payload Example

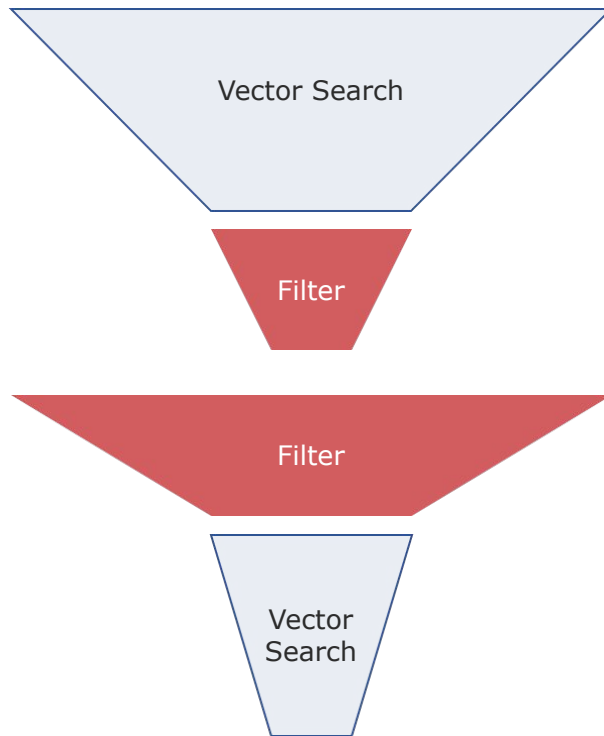
{
  "text": "Nội dung điều khoản...",
  "metadata": {
    "source": "hr_policy.pdf",
    "year": 2026,
    "access": "internal"
  }
}
```

## Post-filtering (Lọc sau)

Search top 100 vector gần nhất, sau đó loại bỏ những kết quả không thuộc năm 2026. **Rủi ro:** Có thể bị sót mất tài liệu quan trọng nếu nó nằm ở hạng 101.

## Pre-filtering (Lọc trước)

Yêu cầu DB chỉ nhìn vào vùng không gian chứa tài liệu năm 2026, rồi mới chạy Vector Search. Nhanh hơn, an toàn hơn, và chính xác tuyệt đối.



Một kịch bản chuẩn bị dữ liệu tiêu chuẩn sử dụng TextSplitter và Vectorstore.

```
# Import các thư viện cần thiết
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma

# Thiết lập cấu trúc quy kèm overlap
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150
)

# Cắt doc và nhồi metadata
chunks = text_splitter.split_documents(raw_documents)
for chunk in chunks:
    chunk.metadata["source"] = "Q1_Report"

# Lưu vào Vector DB
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings
)
```

# Query Processing & Advance Retrieval

---

Moving beyond basic semantic search: How to transform ambiguous user queries, leverage the power of hybrid retrieval, and use cross-encoder re-ranking to extract the highest quality context.

2

## **2.1 Query Transformation**

---

Learn how to intercept, expand, decompose, and transform ambiguous user inputs into highly optimized search queries using LLMs.

## Khoảng cách từ vựng

Người dùng dùng ngôn ngữ "đường phố" hoặc mơ hồ. Tài liệu nội bộ dùng ngôn ngữ chuyên môn trang trọng.

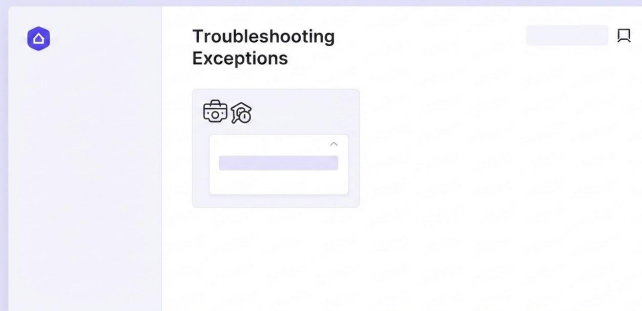
## Thiếu ngữ cảnh

Câu hỏi cụt lủn (vd: "hoàn tiền") khiến Vector DB trả về quá nhiều kết quả nhiễu.

**Kết luận: Nếu lấy raw query đi search thẳng, Retriever sẽ thất bại ngay từ giây đầu tiên.**



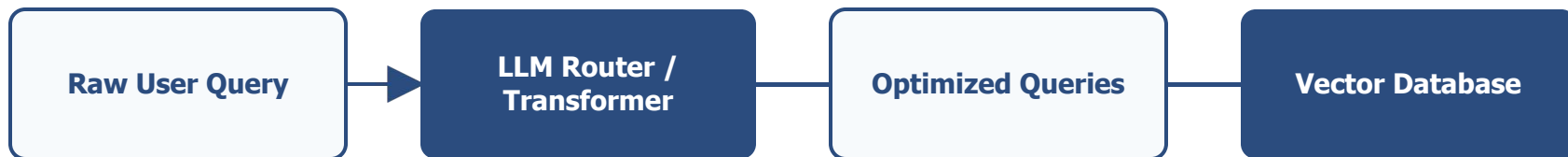
"sao app cứ vắng"



Troubleshooting Unhandled Exceptions



- Đặt một LLM nhỏ, tốc độ cao (như GPT-4o-mini, Gemini Flash) làm "màng lọc" đứng trước Vector Database.
- LLM này đóng vai trò như một người biên tập, diễn dịch lại ý định của người dùng thành các truy vấn tối ưu cho máy học.
- Đây là bước đệm hoàn hảo trước khi ta biến Retriever thành một "Tool" độc lập cho các Agent (LangGraph) ở giai đoạn sau.



- Chữa lỗi chính tả và thêm các từ đồng nghĩa (synonyms) hoặc thuật ngữ chuyên ngành có cùng nghĩa.
- Tăng mạnh độ phủ (Recall), giúp hệ thống không bỏ sót tài liệu chỉ vì người dùng dùng sai từ.

## VÍ DỤ

Raw User Query:  
**"nghỉ đẻ"**



LLM Expanded Queries:  
["nghỉ thai sản", "maternity leave",  
"chế độ phụ sản", "trợ cấp sinh con"]

- Xử lý các câu hỏi phức tạp (Multi-hop). Một Vector biểu diễn tài liệu không thể đồng thời trả lời cho hai ý niệm quá khác biệt.
- Phân tách câu hỏi lớn thành nhiều câu hỏi nhỏ, chạy tìm kiếm song song (parallel retrieval), sau đó gộp context lại.

## VÍ DỤ

Q: "So sánh chính sách hoàn tiền của Shopee và Tiki"

Q1: "Chính sách hoàn tiền Shopee"

Q2: "Chính sách hoàn tiền Tiki"

Gộp Context

- Khi câu hỏi đi quá sâu vào tiểu tiết, model dễ bị "lạc" và không tìm thấy tài liệu chính xác (do quá cụ thể).
- LLM tự động sinh ra một câu hỏi "lùi lại một bước" (abstract/high-level) để lấy được ngữ cảnh quy tắc chung trước khi giải quyết ca cụ thể.

## VÍ DỤ

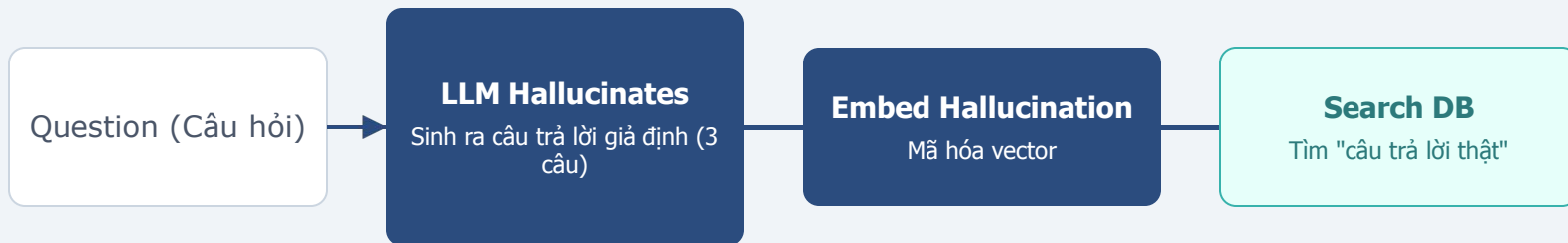
User: "Lỗi 404 khi gọi API thanh toán Momo của user ID 8910"



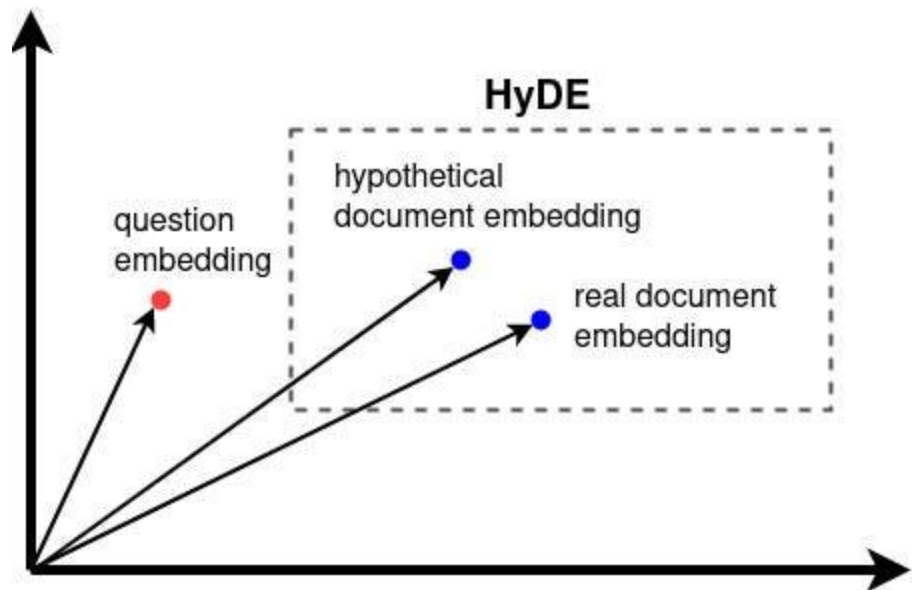
**Step-back Q:**  
"Kiến trúc tích hợp cổng thanh toán Momo hoạt động như thế nào?"

- **Khái niệm:** Dùng LLM "bịa" ra một câu trả lời giả định dựa trên câu hỏi của người dùng.
- **Tại sao?** Vì một đoạn văn bản trả lời (dù sai sự thật) sẽ có cấu trúc ngữ pháp, từ vựng và "hình dáng toán học" cực kỳ giống với tài liệu thật đang nằm trong DB. Ta đem vector của "câu trả lời giả" đi tìm "câu trả lời thật".

## SƠ ĐỒ KHÁI NIỆM



- Vector "Câu hỏi" và "Câu trả lời" thường nằm cách xa nhau (do định dạng ngữ pháp hoàn toàn khác biệt).
- HyDE đóng vai trò như một "phép nội suy", kéo Query Vector về đúng cụm (cluster) không gian chứa Document Vectors.



- Khởi tạo MultiQueryRetriever để tự động sinh nhiều câu hỏi từ 1 prompt gốc.

Python

```
from langchain.retrievers.multi_query import MultiQueryRetriever

prompt_template = "You are an AI assistant. Generate 3 alternative search queries for: {question}"

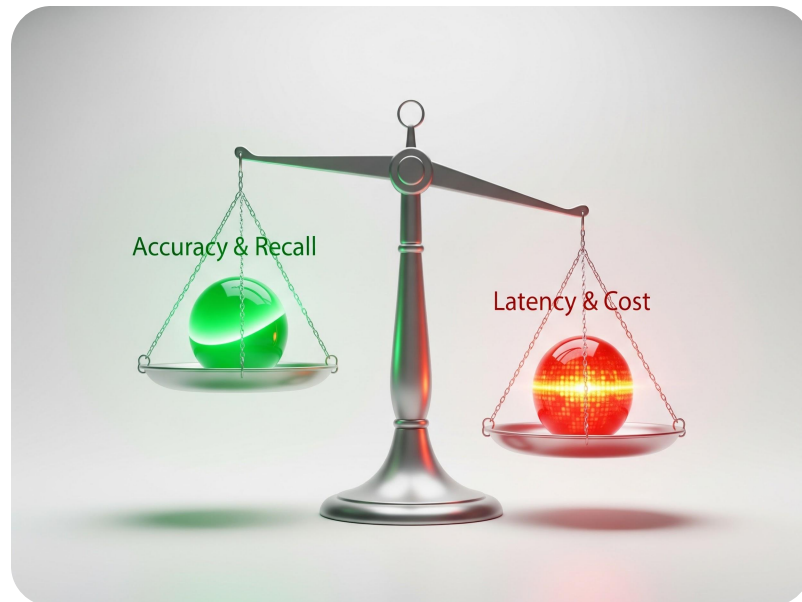
retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(),
    llm=chat_model,
    prompt=prompt_template
)
```

## Được (Benefits)

- Cải thiện cực lớn độ chính xác và Recall.
- Xử lý được các ca người dùng "hỏi ngốc".

## Mất (Costs)

- Tăng độ trễ (Latency) vì phải gọi API LLM 1 lần trước khi đưng vào Database.
- Tốn thêm chi phí (Token cost).





✓ **Đừng lạm dụng!** Chỉ bật Query Transformation khi hệ thống gặp nhiều user queries phức tạp/mơ hồ.

✓ **Dùng model rẻ nhất có thể** (GPT-4o-mini / Haiku) cho bước này để giữ Latency < 1s.

✓ **Kết hợp với Semantic Cache** để không phải transform lại các câu hỏi phổ biến.

## **2.2 Dense vs. Sparse Retrieval**

---

Meaning vs. Keywords: Comparing the semantic understanding of dense vector embeddings against the exact-match precision of sparse retrieval algorithms like BM25.



## Dense Retrieval (Tân binh AI)

Tìm theo "Ý Nghĩa" (Semantic).

Mã hóa văn bản thành mảng vector dày đặc (ví dụ 1536 chiều).



## Sparse Retrieval (Lão làng)

Tìm theo "Từ Khóa".

Dựa trên tần suất xuất hiện của từ (BM25, TF-IDF, Inverted Index).

- Tráit tìm của Elasticsearch và các hệ thống search truyền thống.
- Sparse = Vector rất dài (bằng toàn bộ số từ trong từ điển) nhưng chứa toàn số 0.
- Nguyên lý: Đánh trọng số cực cao cho các từ hiếm (VD: "Kubernetes") và phớt lờ các từ phổ biến (VD: "và", "là", "thì").

## Sparse Vector Representation



Docker

Kubernetes

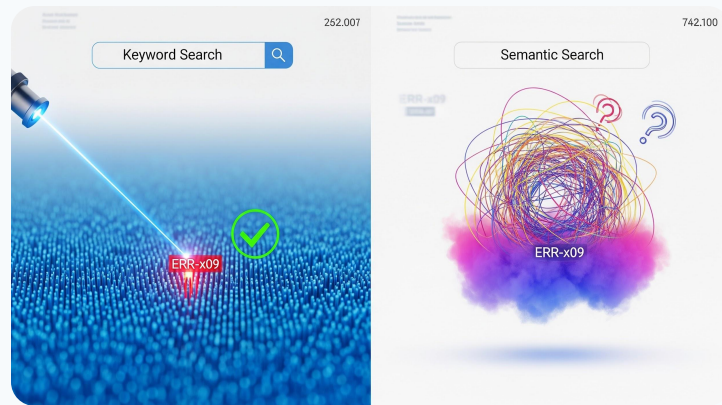
Hầu hết các chiều là 0 (sparse). Chỉ các từ khoá xuất hiện mới có trọng số khác 0.

## Inverted Index Mechanism



## Vô địch Exact Match

- Khớp chính xác: Mã số thuế, ID nhân viên, mã lỗi hệ thống (ERR-x09), từ viết tắt chuyên ngành.
- Vượt qua giới hạn tập train của LLM.
- Vector nhúng thường bị "mù" trước các chuỗi ký tự vô nghĩa hoặc chuyên biệt này.



### CASE STUDY: SEARCH QUERY "ERR-X09"

#### **BM25 (Sparse)**

Bắn trúng ngay tài liệu chứa mã lỗi chính xác.

#### **Vector Search (Dense)**

Loay hoay tìm "ý nghĩa" của x09 và đưa ra kết quả noise.

- Cực kỳ nhạy cảm với lỗi chính tả (Typo). Sai một chữ cái là "Not Found".
- Hoàn toàn không hiểu từ đồng nghĩa (Synonyms) hoặc diễn đạt lại (Paraphrase).

## VISUAL EXAMPLE: SEMANTIC GAP



### **BM25 Result: 0 Results**

User searches: "Tôi muốn đòi lại tiền"

Document: "Chính sách hoàn tiền"

- Bù đắp mọi điểm yếu của BM25. Không quan tâm bạn gõ "hoàn tiền", "trả tiền", hay sai chính tả "**hoang tien**". Nó hiểu "ý niệm" đằng sau chuỗi ký tự.
- Khả năng **Cross-lingual**: User hỏi bằng tiếng Việt, vector search vẫn map đúng vào tài liệu tiếng Anh.

## VISUAL EXAMPLE: DENSE SEARCH POWER

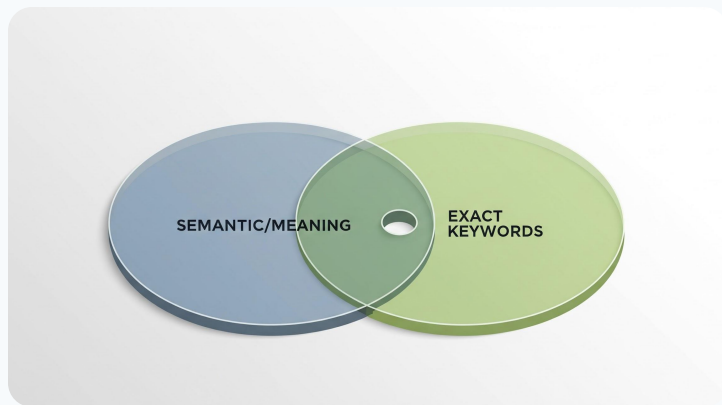


### Vector Search: Success

User searches: "hoang tien" (Typo)

Document Found: "Refund Policy"

- **Vector Search**: Tìm ý nghĩa tốt nhưng **hụt keyword**.
- **BM25**: Tìm keyword tốt nhưng **hụt ý nghĩa**.



## BLIND SPOT ANALYSIS

### Scenario 1: Dense Retrieval (Vector)

Query: "Mã lỗi ERR-x09"

Result: Returns general error handling docs (Misses exact ID).

### Scenario 2: Sparse Retrieval (BM25)

Query: "Muốn lấy lại tiền"

Doc: "Chính sách hoàn trả"

Result: **0 Results** (No keyword match).



| Tiêu chí           | Sparse Retrieval (Truy xuất thưa thớt)                                                                                                                                        | Dense Retrieval (Truy xuất dày đặc)                                                                                                                                                                |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cơ chế cốt lõi     | So khớp từ khóa chính xác (Lexical/Keyword matching). Dựa trên tần suất xuất hiện của từ.                                                                                     | Tìm kiếm theo ngữ nghĩa và ngữ cảnh (Semantic search). Dựa trên khoảng cách giữa các vector.                                                                                                       |
| Biểu diễn Vector   | Vector có số chiều rất lớn (bằng kích thước toàn bộ từ vựng), chứa chủ yếu là các giá trị 0 (sparse).                                                                         | Vector có số chiều thấp và cố định (VD: 384, 768, 1536 chiều), chứa các số thực (dense).                                                                                                           |
| Thuật toán/Mô hình | TF-IDF, BM25.                                                                                                                                                                 | Các mô hình nhúng (Embedding Models) như BERT, OpenAI Embeddings, Cohere.                                                                                                                          |
| Điểm mạnh          | - Tốc độ tính toán nhanh, chi phí phần cứng thấp.- Hiệu quả tuyệt đối với các từ khóa hiếm, tên riêng, mã định danh (ID), hoặc các thuật ngữ chuyên ngành đặc thù.            | - Hiểu được từ đồng nghĩa, khái niệm tương đương và cấu trúc câu.- Truy xuất tốt ngay cả khi câu hỏi của người dùng và tài liệu gốc không dùng chung hệ thống từ vựng.                             |
| Điểm yếu           | - Không hiểu được ý nghĩa của câu (sẽ thất bại nếu người dùng sử dụng từ đồng nghĩa hoặc cách diễn đạt khác).- Dễ bị nhiễu bởi các từ phổ biến nếu không lọc kỹ (stop words). | - Đòi hỏi tài nguyên tính toán cao hơn (thường cần GPU để tạo embedding ở quy mô lớn).- Yêu cầu hạ tầng chuyên dụng như Vector Database và các thuật toán tìm kiếm xấp xỉ (ANN) để đảm bảo tốc độ. |
| Ứng dụng thực tiễn | Tìm kiếm chính xác các mã lỗi, tên khách hàng cụ thể, hoặc các tham số kỹ thuật trong tài liệu.                                                                               | Nhận diện ý định và trả lời các câu hỏi tự nhiên phức tạp của người dùng trong các hệ thống hỏi đáp (Q&A).                                                                                         |

- Các tutorial YouTube chỉ dạy bạn **Vector Search** vì nó "nghe có vẻ AI".
- Ở môi trường Doanh nghiệp (tra cứu hợp đồng, log kỹ thuật, tài liệu luật), **BM25** thường quan trọng hơn Vector.
- **Nếu bỏ BM25, hệ thống sẽ thất bại thảm hại.**



## 2.3 Hybrid Search Deep Dive

---

The best of both worlds: A deep dive into Hybrid Search, combining dense semantic vectors with sparse exact-match keywords using Reciprocal Rank Fusion (RRF) and Alpha-tuning.

- **Khái niệm:** Chạy song song cả BM25 và Vector Search cho cùng một câu hỏi.
- Đảm bảo hệ thống không bỏ lỡ mã số (nhờ **BM25**) và cũng không lọt ngữ nghĩa (nhờ **Vector**).
- Trở thành **tiêu chuẩn bắt buộc** cho Production RAG hiện đại.



## Vector Score

Cosine Similarity thường nằm trong khoảng

**0.0** → **1.0**

## BM25 Score

Không có giới hạn trên, có thể từ **0** → **100+**

## Thách thức

Không thể cộng trực tiếp **Score\_Vector** + **Score\_BM25** để xếp hạng.

- **Đừng gộp điểm số (Scores)**, hãy gộp **Thứ hạng (Ranks)**.

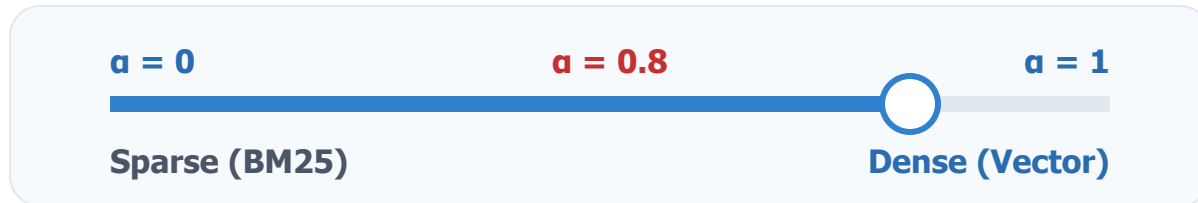
$$\text{RRF} = 1 / (k + \text{Rank\_Dense}) + 1 / (k + \text{Rank\_Sparse})$$

*(Giá trị hằng số k thường được chọn mặc định là 60)*

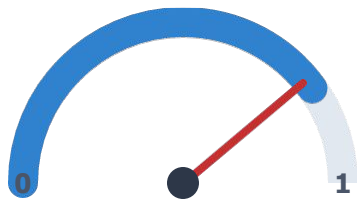
- Tài liệu nào nằm trong **Top cao ở cả 2 bảng xếp hạng** sẽ vươn lên vị trí số 1 tuyệt đối.

- Kiểm soát hệ thống nghiêng về bên nào thông qua trọng số  $\alpha$  (0.0 đến 1.0).
- Nếu  $\alpha = 1$ : Thuần Vector. Nếu  $\alpha = 0$ : Thuần BM25.

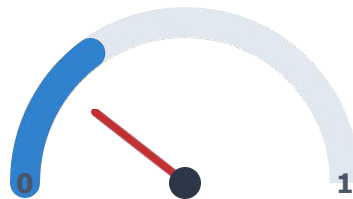
$$\text{Final\_Score} = (\alpha \times \text{Dense\_Score\_norm}) + ((1 - \alpha) \times \text{Sparse\_Score\_norm})$$



- **Hệ thống Chatbot FAQ:**  $\alpha = 0.7 - 0.9$  (Ưu tiên hiểu ý định mơ hồ của người dùng).
- **Tra cứu Code, Log, Luật pháp:**  $\alpha = 0.2 - 0.4$  (Ưu tiên khớp chính xác tên biến, điều khoản).



**Chatbot FAQ**  
 $\alpha = 0.8$  (Semantic)

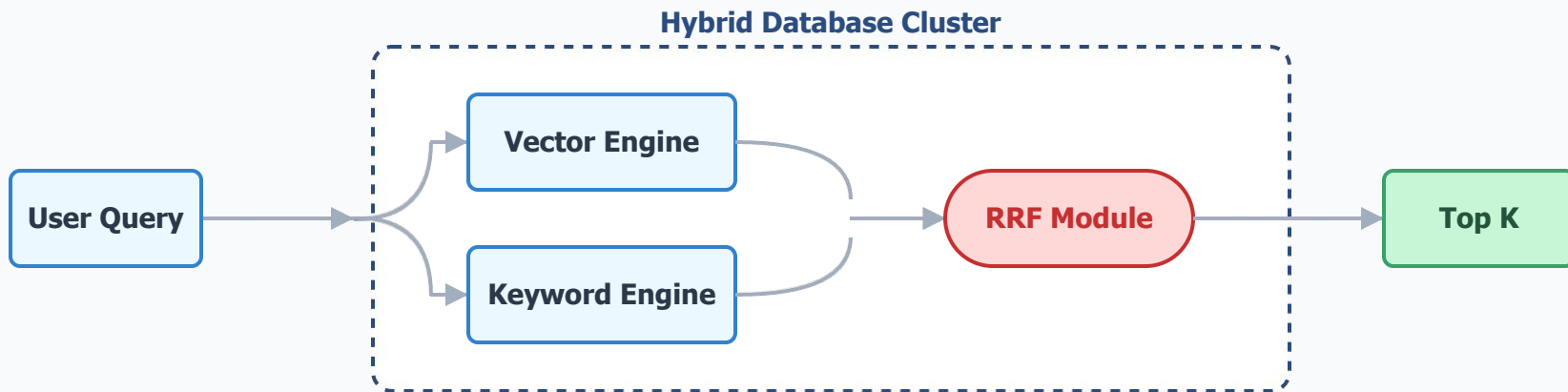


**Code / Luật pháp**  
 $\alpha = 0.3$  (Keyword)



- Không phải Vector DB nào cũng hỗ trợ Hybrid chuẩn. Cần database hỗ trợ lưu cả Dense vectors và Sparse/Inverted indexes cùng lúc.

**Các hệ thống nổi bật hiện nay:** Weaviate, Milvus, Qdrant, Elasticsearch (kết hợp plugin).



- Khởi tạo **Hybrid Retriever** trong LangChain sử dụng thuật toán gộp.

```
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever

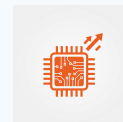
# Khởi tạo 2 bộ máy đọc lập
bm25_retriever = BM25Retriever.from_documents(docs)
vector_retriever = faiss_index.as_retriever()

# Gộp lại với trọng số Alpha (VD: 30% Keyword, 70% Vector)
hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, vector_retriever],
    weights=[0.3, 0.7]
)
```



## Lưu trữ (Storage)

Phải nhân đôi tài nguyên lưu trữ (Build 2 index khác nhau cho cùng 1 tập dữ liệu).



## Hiệu năng (CPU Load)

Tăng tải CPU khi truy vấn do phải chạy 2 thuật toán song song.

**Sự đánh đổi xứng đáng cho chất lượng Enterprise.**

## 2.4 Re-ranking

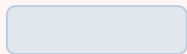
---

Broad search is not enough. Discover how cross-encoders score relevance with sniper precision, and how MMR eliminates redundant context to optimize your LLM's token budget.

## k quá thấp

**Ví dụ:** top-1 hoặc top-2

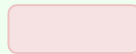
**Triệu chứng:** thiếu chứng cứ, recall kém



## Sweet spot

**Ví dụ:** top-3 đến top-5

**Triệu chứng:** đủ chứng cứ, ít nhiễu



## k quá cao

**Ví dụ:** top-10 trở lên

**Triệu chứng:** context nhiễu, token lãng phí



**Lưu ý:** Mục tiêu của retrieval không phải là **lấy nhiều**, mà là lấy **đúng và đủ** cho generation.

## Đặc tính Retriever

- Retriever (Vector/Hybrid) được thiết kế để quét qua hàng triệu tài liệu cực nhanh. Nó đánh giá sự liên quan một cách "thô và rộng".
- Hệ quả: Chứa rất nhiều ngữ cảnh tương đồng nhưng không trực tiếp trả lời câu hỏi. Tài liệu đúng nhất có thể đang nằm ở Top 10, chứ không phải Top 1.

## Trực quan: Query & Results

**Query: "Thủ tục xin visa"**

**Top 1:** Giá làm visa

**Top 2:** Lịch sử visa

...

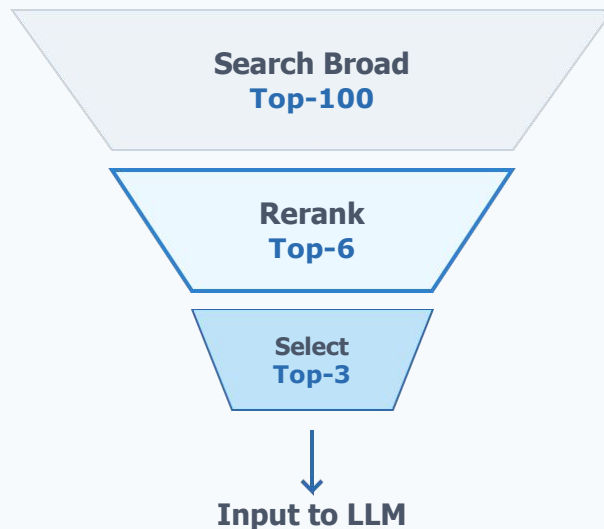
**Top 8:** Các bước làm thủ tục

**Dữ liệu đúng bị "chìm" sâu**

## Quy trình xử lý

- Giai đoạn 1: Dùng Hybrid Search kéo nhanh về Top 50-100 tài liệu. Nhanh nhưng nhiễu.
- Giai đoạn 2: Đưa Top 50 này qua một mô hình AI khác (Re-ranker) để đọc kỹ và chấm điểm lại sự liên quan. Lấy Top 3-5 đưa cho LLM.

## Rerank Funnel Visual

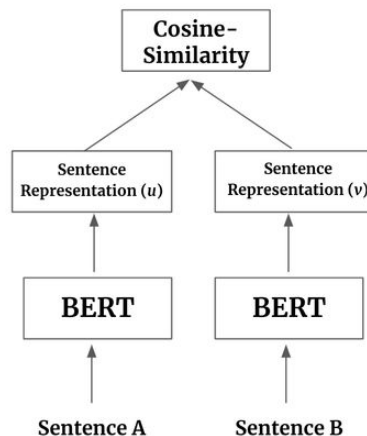


## Bi-Encoder (Vector DB)

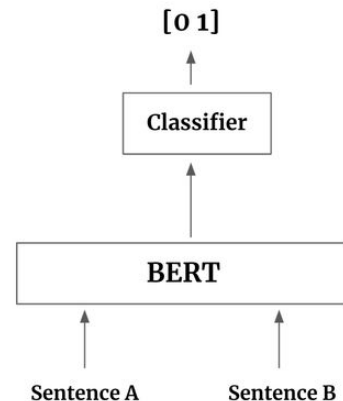
- Query và Document đi qua hai luồng riêng biệt.
- Chỉ tính khoảng cách lúc cuối. Rất nhanh.

## Cross-Encoder (Reranker)

- Gắn Query và Document thành một đoạn text duy nhất (Query + [SEP] + Document).
- Cho qua Transformer cùng lúc. Nhờ cơ chế Attention, model hiểu chính xác sự tương tác giữa từng từ.



**Bi-Encoder**

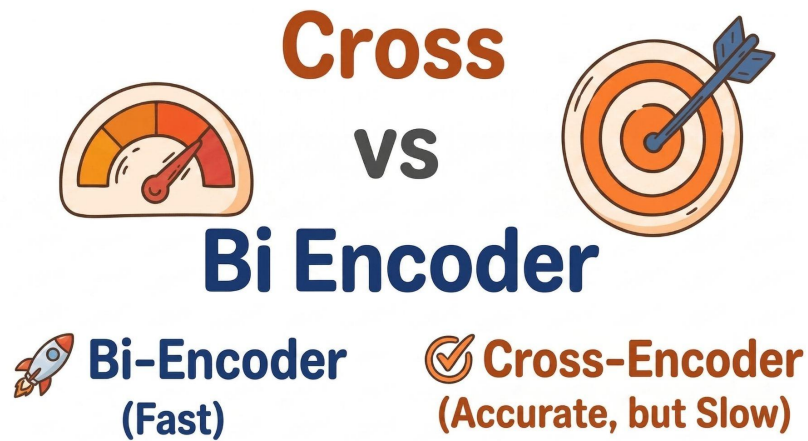


**Cross-Encoder**



## Đặc điểm mô hình

- Cross-Encoder chấm điểm cực kỳ chính xác (như một người đọc kiểm tra chéo).
- Nhưng nó quá chậm và tốn compute. Không bao giờ được dùng Reranker để quét toàn bộ database.
- Chỉ dùng cho list nhỏ đã lọt qua vòng 1.



## Hạn chế của Re-ranker

- Re-ranker có thể đưa Top 3 tài liệu tốt nhất lên đầu. Nhưng nếu cả 3 tài liệu này đều sao chép nội dung của nhau thì sao?
- LLM sẽ tốn token vô ích mà không có thêm góc nhìn hay dữ kiện mới.

## Minh họa Redundancy

### Chunk 1

"Hoàn tiền mất 7 ngày."

### Chunk 2

"Tiền sẽ về sau 7 ngày."

### Chunk 3

"Thời gian xử lý hoàn tiền là 7 ngày."

**Redundant Context Window!**

## Cơ chế hoạt động của MMR

- **MMR (Maximum Marginal Relevance):** Thuật toán chọn lọc để tối đa hóa sự liên quan (Relevance) nhưng phạt nặng sự trùng lặp (Redundancy).
- **Bước 1:** Chọn chunk liên quan nhất với Query.
- **Bước 2:** Chọn chunk tiếp theo vừa liên quan Query, vừa có khoảng cách vector xa nhất so với chunk số 1.

## Công thức MMR

**Maximize:**  $[Similarity(Doc, Query)] - Penalty \times [Similarity(Doc, Already\_Selected\_Docs)]$

## Precision Focus

### Cross-Encoder

Dùng cho các câu hỏi cần sự chính xác tuyệt đối (Fact-checking, Legal).

#### Ví dụ:

"Các yêu cầu an toàn khi lắp đặt máy phát điện là gì, và những lời khuyên về bảo trì là gì?"

## Diversity Focus

### MMR

Dùng cho các truy vấn mở, cần tổng hợp nhiều góc nhìn.

#### Ví dụ:

"Hãy tóm tắt các điểm rủi ro của dự án A từ tất cả các báo cáo"

Sử dụng Re-ranker as a Service (như Cohere) là cách tiết kiệm tài nguyên hệ thống nhất.

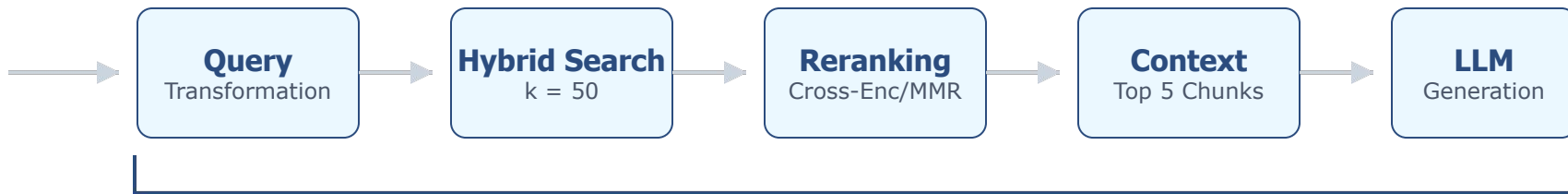
```
# Python

from langchain.retrievers import ContextualCompressionRetriever
from langchain_cohere import CohereRerank

# Gọi API Reranker (VD model multilingual cho tiếng Việt)
compressor = CohereRerank(top_n=3, model="rerank-multilingual-v3.0")

# Bọc retriever cũ bằng layer rerank
rerank_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=hybrid_retriever
)
```

Tổng hợp toàn bộ Module 2: Lộ trình từ Query đầu vào đến kết quả cuối cùng thông qua các kỹ thuật tối ưu hóa retrieval đã học.



**Wrapped as an Agent Tool**

**Gợi ý**

Ta có thể gói gọn pipeline này thành công cụ cho Agentic System (LangGraph) gọi tự động.

## **Generation, Grounding & UX**

The final mile of RAG: Mastering context injection, enforcing strict LLM grounding to eliminate hallucinations, and crafting an output UX that users actually trust.

3

## 3.1 Context Injection Patterns

---

The art of context injection: How to structure and format retrieved data within the LLM's prompt window to maximize retention and conquer the 'Lost in the Middle' effect.



Nếu Indexing là "xây kho", Retrieval là "người thủ thư tìm sách", thì Generation là "biên tập viên" tổng hợp thông tin.

Dù bạn tìm được tài liệu xuất sắc đến đâu, nếu không biết cách "bơm" (inject) nó vào Prompt, LLM vẫn sẽ bị ảo giác hoặc trả lời sai định dạng.



Là nghệ thuật sắp xếp và định dạng các chunk dữ liệu (đã retrieve) vào trong Context Window của LLM để nó dễ đọc, dễ hiểu nhất.

Không chỉ là nối chuỗi (text1 + text2).  
Cách bạn định dạng quyết định việc LLM có tôn trọng dữ liệu đó hay không.

## THE PROMPT

[System Rules]

[Retrieved Documents]

[User Question]

Cách làm sơ khai nhất: Ghép tất cả các chunk thành một khối văn bản dài và đặt lên đầu câu hỏi.

- **Ưu điểm:** Dễ code (chỉ cần hàm `.join()`).
- **Nhược điểm:** Model không phân biệt được ranh giới giữa các tài liệu, làm mất giá trị Metadata.

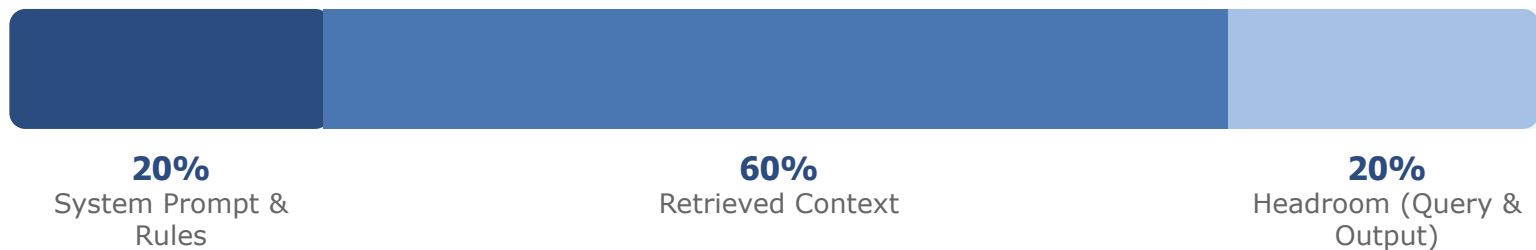


Cách làm chuẩn Production: Sử dụng thẻ XML hoặc định dạng JSON/Markdown để phân tách rõ ràng từng nguồn dữ liệu.

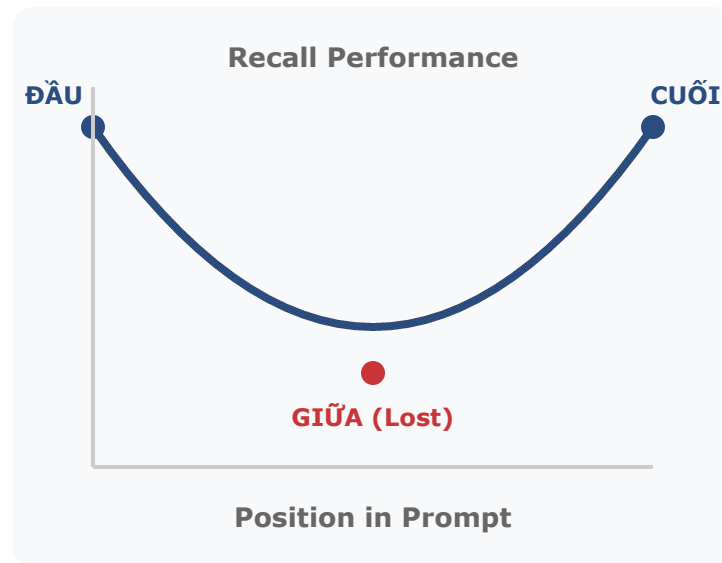
Nhờ cấu trúc này, model dễ dàng nhận diện ID của tài liệu để làm trích dẫn (Citation) sau này.

```
<documents>
  <doc id="1" source="policy.pdf">
    ... content document 1 ...
  </doc>
  <doc id="2">
    ... content document 2 ...
  </doc>
</documents>
```

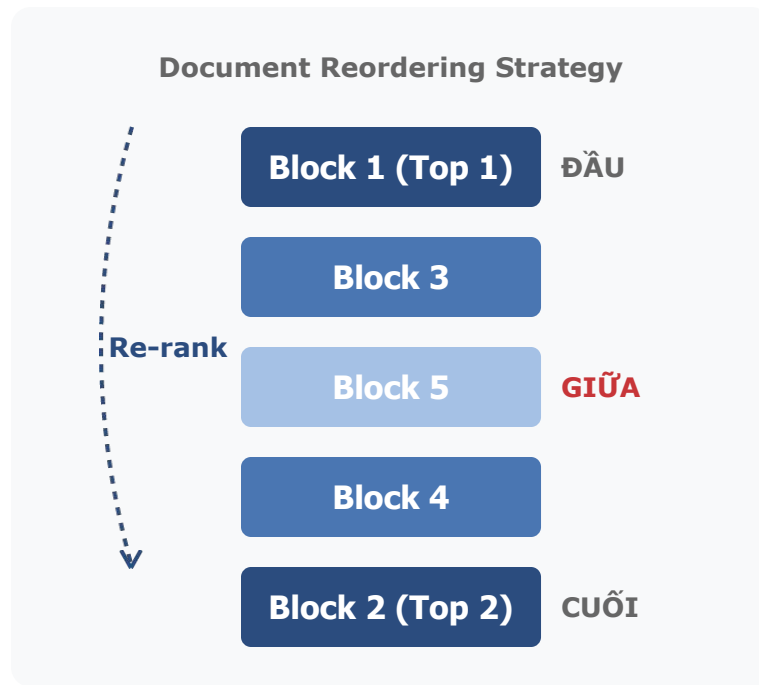
- Đừng nhồi tối đa Token chỉ vì model hỗ trợ. Càng nhiều context → Càng chậm → Càng đắt → Càng dễ nhiễu.
- Phải chia ngân sách rõ ràng: 20% cho System Prompt/Rules, 60% cho Retrieved Context, 20% dự phòng (Headroom) cho User Query và Output.



- Các nghiên cứu chỉ ra: LLM giống như con người, nó nhớ rất tốt thông tin nằm ở ĐẦU và CUỐI prompt, nhưng thường "bỏ quên" thông tin nằm ở GIỮA nếu prompt quá dài.
- Nếu chunk chứa câu trả lời quan trọng nhất vô tình bị xếp ở giữa danh sách, RAG có thể thất bại.



- Đừng ném nguyên Top K từ Reranker vào prompt theo thứ tự 1, 2, 3, 4, 5.
- Thủ thuật Document Reordering: Sắp xếp lại theo mẫu luân phiên. Đặt tài liệu tốt nhất ở đầu, tốt thứ 2 ở cuối, các tài liệu điểm thấp giấu vào giữa.
- Thứ tự đưa vào prompt: [1, 3, 5, 4, 2]



## **3.2 Prompt Engineering for Strict Grounding**

Taming the LLM: Discover how to construct robust system prompts that enforce strict citations, prevent hallucinations, and gracefully handle knowledge gaps.



- Grounding (Tiếp đất / Neo dữ kiện) là việc bắt buộc LLM chỉ được phép sử dụng thông tin từ context được cấp, nghiêm cấm dùng "kiến thức học được từ Internet" để chém gió.
- Mục tiêu: Nếu đổi context sai, model cũng phải trả lời sai theo context đó. Trọng tài duy nhất là dữ liệu nội bộ.



Một System Prompt tốt cho RAG cần 4 phần cốt lõi:

## 1. Role

Định hình nhân vật  
(VD: "Bạn là trợ lý pháp chế nội bộ...").

## 2. Task

Trả lời câu hỏi người dùng.

## 3. Context

Cung cấp tài liệu từ hệ thống Retrieval.

## 4. Strict Constraints

Quy tắc "bàn tay sắt"  
(Cấm bịa, bắt buộc trích dẫn).

- RAG mất đi 50% giá trị nếu model không chỉ ra được nó lấy câu trả lời từ dòng nào, tài liệu nào.
- Trong Constraint, cần chỉ thị rõ: **"Khi đưa ra một tuyên bố, phải trích dẫn ID của document trong ngoặc vuông, ví dụ [doc\_1]."**

## PROMPT SNIPPET

ALWAYS cite your sources using the `<doc_id>` provided.



## RESULTING OUTPUT

"Nhân viên được nghỉ 12 ngày phép **[doc\_3]**."

- Đây là tính năng quan trọng nhất của RAG: Biết giới hạn của mình (Graceful Degradation).
- Nếu Top K chunks mang về không chứa câu trả lời, model phải từ chối thay vì cố gắng đoán mò.

## PROMPT LINE

*If the context does not contain the answer, reply EXACTLY with: "Dữ liệu hiện tại không đủ để tôi trả lời câu hỏi này." Do not attempt to guess.*

- Thay vì chỉ nói "Không biết" cụt lủn gây ức chế cho người dùng.
- Hãy prompt để model gợi ý: "Tôi không tìm thấy chính sách này trong kho tài liệu HR năm 2026. Bạn có muốn tôi tìm kiếm rộng hơn hoặc liên hệ bộ phận nhân sự không?"



**BAD CHATBOT**

"Tôi không biết."

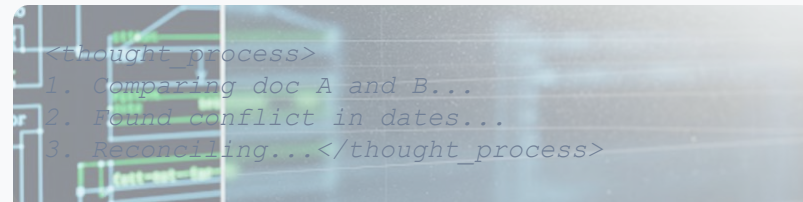


**GOOD CHATBOT**

*"Tôi chưa tìm thấy [X], nhưng bạn có thể thử hỏi lại với từ khóa [Y] hoặc tạo ticket cho IT."*

- Yêu cầu model "suy nghĩ ra nháp" trước khi in ra câu trả lời cuối.
- Chỉ thị: "Đầu tiên, hãy lọc ra các câu liên quan trong context. Phân tích chúng trong thẻ <thought\_process>. Sau đó mới tổng hợp thành câu trả lời."
- Tăng độ chính xác lên đáng kể đối với các câu hỏi so sánh hoặc suy luận logic từ tài liệu.

## INTERNAL REASONING



```
<thought_process>
1. Comparing doc A and B...
2. Found conflict in dates...
3. Reconciling...</thought_process>
```

## FINAL ANSWER

Dựa trên phân tích tài liệu, câu trả lời chính xác nhất là phương án B vì các lý do sau đây: ...

- Cách lắp ghép linh động các thành phần Context và User Question vào trong Prompt.

## PYTHON SNIPPET

```
rag_prompt_template = """
You are a strictly grounded assistant.
Answer the user's question using ONLY the context below.
If you cannot answer, say "I don't know".
Always cite the [source_id].

<context>
{formatted_context}
</context>

Question: {question}
Answer:
"""
```

## **3.3 Output Formatting & UX**

---

Learn how to design an enterprise-grade Output UX, featuring inline citations, transparent source blocks, and fluid streaming states that build user trust.



Người dùng doanh nghiệp không quan tâm bạn dùng thuật toán MMR hay HNSW. Họ chỉ nhìn vào giao diện cuối cùng. Một khối text đặc chữ sẽ tạo cảm giác lười đọc và nghi ngờ. Cần thiết kế đầu ra có tính "scannable" (dễ quét mắt).

## CÁCH CŨ: KHÓ THEO DÕI

Dựa trên báo cáo quý 3, doanh thu đạt 5.2 tỷ USD, tăng 12% so với cùng kỳ năm ngoái nhờ vào việc tối ưu hóa chi phí vận hành tại khu vực Đông Nam Á trong khi đó tỷ lệ giữ chân khách hàng vẫn duy trì ở mức 85% mặc dù có sự cạnh tranh gay gắt từ các đối thủ mới nổi và kế hoạch cho quý 4 sẽ tập trung vào việc mở rộng mảng dịch vụ đám mây với mục tiêu tăng trưởng thêm 15% thông qua các gói ưu đãi dành cho khách hàng trung thành đã sử dụng dịch vụ trên 2 năm.

## CÁCH MỚI: SCANNABLE UX

### Kết quả kinh doanh Quý 3:

- **Doanh thu:** 5.2 tỷ USD (+12% YoY)
- **Động lực chính:** Tối ưu vận hành tại Đông Nam Á.
- **Khách hàng:** Tỷ lệ giữ chân ổn định ở mức **85%**.

### Kế hoạch Quý 4:

- Mở rộng dịch vụ đám mây (Mục tiêu **+15%**).
- Ưu đãi cho khách hàng trung thành (>2 năm).

- **Giống Wikipedia:** Đặt các reference ID ngay sát bên cạnh thông tin kiện.
- **Góc độ UI/UX:** Các ID này (ví dụ [1], [2]) nên là hyperlink. Khi hover/click vào, nó sẽ popup ra đoạn text gốc để user đối chiếu nhanh.

## AI ASSISTANT MOCKUP

Hoàn tiền diễn ra trong 7 ngày **[1]**

**Source: Refund\_Policy.pdf**

Section 3: All verified claims are processed within 7 business days.

- Ở cuối mỗi câu trả lời, luôn tổng hợp lại một danh sách các tài liệu đã được sử dụng.
- Cung cấp URL hoặc nút "Mở tài liệu gốc" để người dùng đi sâu vào nghiên cứu nếu cần.

## AI ASSISTANT MOCKUP

Theo tài liệu v4.0, thiết bị của bạn được bảo hành 12 tháng kể từ ngày kích hoạt.

### **Nguồn tham khảo:**

1. Chính sách bảo hành v4.0 (Tỷ lệ khớp: 92%)
2. Ticket lỗi #8892 - Jira

→ **Mở tài liệu gốc**

- Đưa thêm tín hiệu từ hệ thống Retrieval ra thẳng UI.
- Nếu điểm số Re-ranker thấp: **Gán nhãn cảnh báo độ liên quan.**
- Giúp quản lý kỳ vọng của người dùng.

## AI ASSISTANT MOCKUP

Dựa trên các tài liệu tìm thấy, quy trình hoàn tiền mất khoảng 7 ngày làm việc.

### CẢNH BÁO ĐỘ TIN CẬY THẤP

Dữ liệu tìm thấy có độ liên quan thấp, câu trả lời có thể không chính xác.

- Hệ thống RAG chạy qua nhiều bước thường mất 3-5 giây, dễ làm user tưởng app bị treo.
- Giải pháp UX: Hiển thị các bước đang chạy và dùng chế độ Streaming khi có text.

## AI ASSISTANT MOCKUP



*Đang tìm kiếm trong kho HR...*



**Đang đọc 5 tài liệu...**

**Dựa trên chính sách nghỉ phép năm 2026...**

Nhân viên có hơn 3 năm thâm niên được hưởng 15 ngày phép |

## 3.4 Generation Failures

---

Learn to diagnose and troubleshoot common generation failures, from conflicting documents to dangerous LLM over-extrapolation.

- Không phải lúc nào lỗi cũng do DB. Có những lúc hệ thống tìm về đúng tài liệu xuất sắc, nhưng LLM vẫn "vấp ngã" ở bước cuối.
- Đây là lúc phải Debug Prompt và Temperature của model.

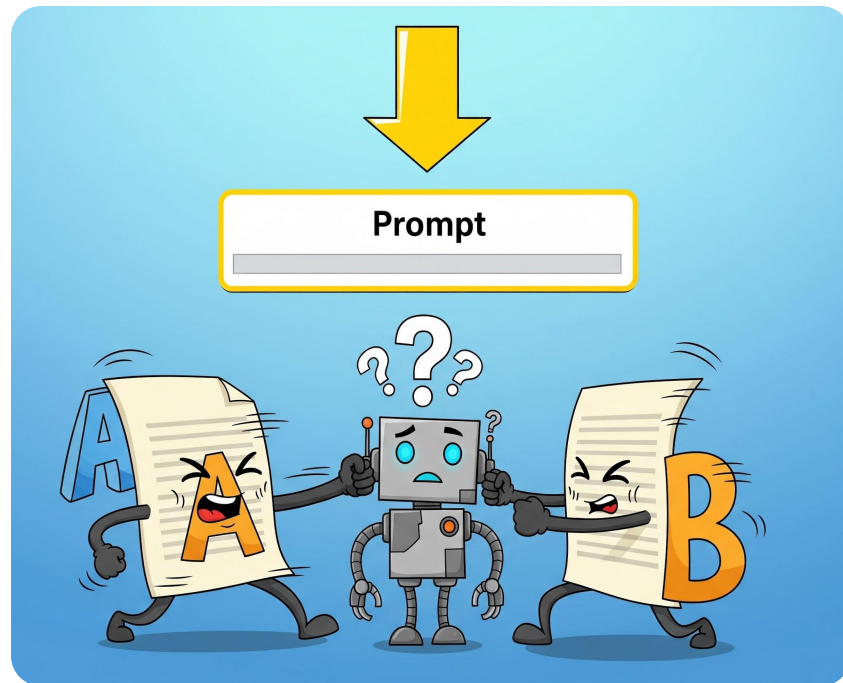


**"Good Context + Bad Prompt  
= Bad Answer"**

**Tình huống:** Retriever mang về 2 tài liệu. Tài liệu A (năm 2024) bảo nghỉ 12 ngày. Tài liệu B (năm 2026) bảo nghỉ 14 ngày.

**Hệ quả:** LLM bị bối rối, có thể cộng gộp, báo lỗi, hoặc chọn bừa.

**Khắc phục:** Dặn dò trong prompt: "Nếu có mâu thuẫn, ưu tiên tài liệu có ngày cập nhật mới nhất, hoặc liệt kê cả 2 và chỉ ra sự mâu thuẫn."



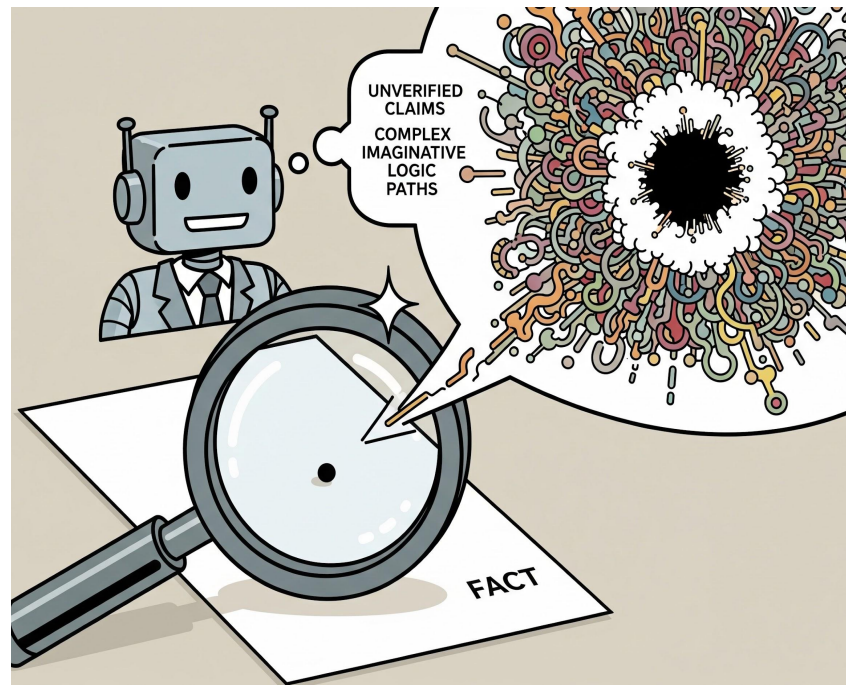


# Lỗi 2 - Over-extrapolation (Suy Diễn Quá Đà)

**Tình huống:** Tài liệu ghi "Miễn phí ship cho đơn trên 500k ở Hà Nội". Người dùng hỏi "Thế ở HCM thì sao?".

**Hệ quả:** Tài liệu không nói về HCM, nhưng LLM tự suy diễn logic "Hà Nội được thì HCM chắc cũng được" → Ảo giác (Hallucination).

**Khắc phục:** Strict Grounding: "Không tự ý suy luận các điều kiện không được đề cập rõ ràng."

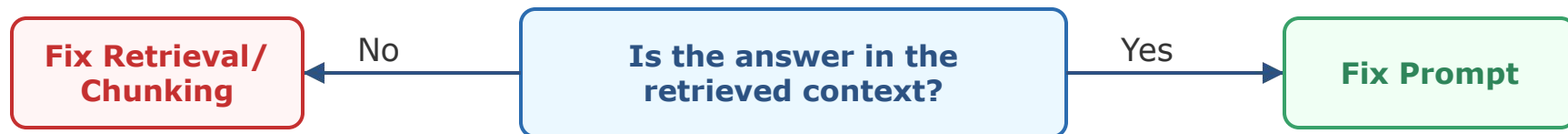


**Tình huống:** Đã dặn model phải trích dẫn ID, nhưng khi sinh ra text, model quên bằng mất tiêu. Thường xảy ra với model nhỏ (ví dụ 8B tham số) hoặc khi context quá dài.

## Khắc phục:

- Đặt các rule quan trọng nhất ở CUỐI prompt (gần chữ Answer: nhất).
- Giảm tham số temperature = 0 (làm cho output deterministic và bớt sáng tạo).

- Nếu app trả lời sai, hãy in log (print) biến **formatted\_context** ra console trước tiên.
- Nếu context có chứa đáp án → **Lỗi do Generation** (Sửa prompt, thêm CoT, đổi model).
- Nếu context KHÔNG chứa đáp án → **Lỗi do Retrieval** (Quay lại Phần 2).



# **Evaluation, Production & Next Steps**

Learn how to quantitatively evaluate your RAG pipeline using the Evaluation Triad, and preview the transition into complex, multi-agent workflows.

# 4

## 4.1 The RAG Evaluation Triad

---

Discover the RAG Evaluation Triad—Context Recall, Faithfulness, and Answer Relevance—to quantitatively measure and debug your system's true performance.

"Vibe check": Nhập thử 3-5 câu hỏi, thấy mượt mà rồi kết luận Ready. **Đây là cái bẫy chết người!**

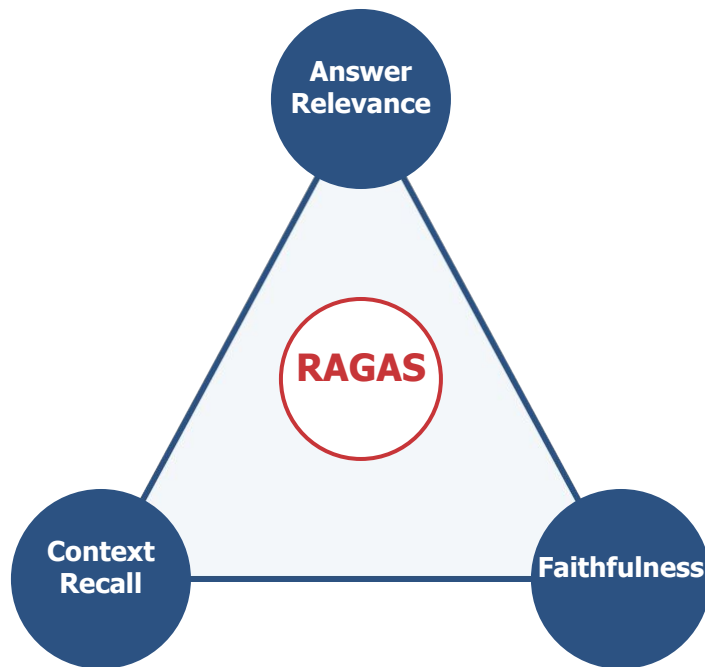
Thay đổi nhỏ (ví dụ chunk 1000 → 500) có thể tốt cho 10 câu này, nhưng lại **làm hỏng 100 câu khác.**

**Phải có Framework định lượng  
(Automated Metrics)**



Hình minh họa: Rủi ro khi chỉ kiểm thử thủ công

- Không thể chấm điểm RAG bằng 1 con số duy nhất. Phải tách bạch lỗi do **Retriever** (tìm sai) hay lỗi do **Generator** (nói bậy).
- Khung RAGAS chia thành 3 trục cốt lõi (**The Triad**): Context Recall, Faithfulness, và Answer Relevance.



## Định nghĩa

Retriever có mang về đủ thông tin cần thiết để trả lời trọn vẹn câu hỏi không?

## Bài toán

Nếu câu hỏi cần 3 chứng cứ (A, B, C), nhưng hệ thống chỉ tìm được A và B  $\rightarrow$  Recall thấp.

## Cách khắc phục

- Tối ưu hóa Vector DB
- Dùng Hybrid Search
- Tăng Top-K



## Định nghĩa

Câu trả lời có bám sát 100% vào tài liệu không, hay đang tự bịa thêm (**Hallucinate**)?

## Nguyên lý

Nếu Context là "A", mà LLM trả lời "A + B", độ trung thực sẽ bị **trừ điểm nặng nề**.

## Cách khắc phục

- Tinh chỉnh System Prompt
- Ép buộc trích dẫn (Citations)
- Giảm Temperature về 0

## Ví dụ

### Context:

"Sản phẩm A màu đỏ."

### LLM Answer:

"Sản phẩm A màu đỏ và ~~được bảo hành 1 năm.~~"

 **Lỗi Hallucination (Bịa thông tin)**

## Định nghĩa

Câu trả lời có đi thẳng vào vấn đề người dùng hỏi không, hay đang trả lời vòng vo, dông dài?

## Vấn đề

Đôi khi LLM trung thực với Context, nhưng Context lại không liên quan đến câu hỏi, dẫn đến một câu trả lời "đúng sự thật nhưng vô dụng".

## Ví dụ

### User Question:

"Thời gian bảo hành của sản phẩm A?"



### LLM Answer:

"Sản phẩm A này rất tốt và có màu xanh, ..."



**Trả lời không đúng trọng tâm (Low Relevance)**

## Vấn đề & Giải pháp

- Làm sao để tính được 3 điểm số trên tự động cho hàng nghìn câu hỏi?  
**Con người không thể ngồi đọc tay.**
- Sử dụng một LLM "Thầy Giáo" (phải là model rất mạnh như **GPT-4o** hoặc **Claude 3.5 Sonnet**) để đọc và chấm điểm LLM "Học Sinh" (hệ thống RAG của bạn).

### LLM-as-a-Judge



## Chuẩn bị dữ liệu

Để chạy RAGAS, bạn cần chuẩn bị một **File Excel/CSV** chứa khoảng **50-100 mẫu thử cực tốt**.

## Các cột bắt buộc

- **Question:** Câu hỏi
- **Ground Truth:** Câu trả lời đúng mà con người kỳ vọng
- **Contexts:** Tài liệu gốc chứa đáp án

## Yêu cầu nội dung

Phải bao gồm đa dạng các loại câu hỏi để đánh giá toàn diện hệ thống:

- Câu hỏi đánh đố
- Câu hỏi mơ hồ
- Câu hỏi không có trong tài liệu

Ví dụ Python cách nạp dữ liệu và xuất bảng điểm báo cáo tự động.

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevance, context_recall
# dataset chứa câu hỏi, câu trả lời sinh ra, và context
result = evaluate(
    dataset,
    metrics=[context_recall, faithfulness, answer_relevance],
    llm=gpt4_judge)

print(result.to_pandas())
```

## **4.2 A/B Testing & Scorecards**

---

Learn how to isolate variables through rigorous A/B testing and interpret evaluation scorecards to drive data-backed improvements in your RAG pipeline.

## Nguyên tắc sống còn

Chỉ thay đổi **MỘT biến số** trong mỗi lần thử nghiệm (A/B Test).

Nếu bạn vừa đổi kích thước chunk, vừa đổi thuật toán Hybrid, vừa đổi System Prompt →  
*Không biết chính xác yếu tố nào mang lại thành công.*

## Trình bày & So sánh

- Trình bày kết quả trực quan cho các Stakeholder (Sếp/PM) xem.
- So sánh điểm số trung bình của toàn bộ 100 câu test trước và sau khi thay đổi thuật toán.

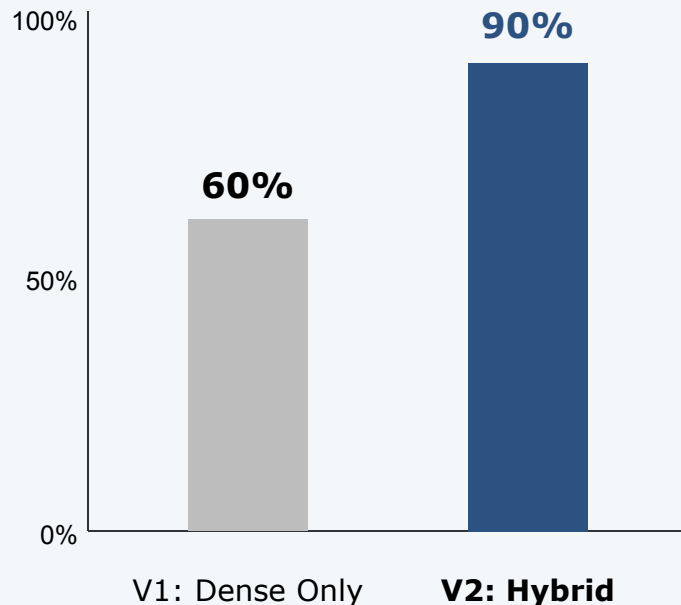
## Ví dụ thực tế

Chuyển từ Thuần Vector (Dense) sang **Hybrid (BM25 + Vector)**.

## Kết quả trên bảng điểm

- Context Recall tăng vọt từ **60% lên 90%** (vì bắt được các mã lỗi chính xác).
- Kéo theo Faithfulness tăng.

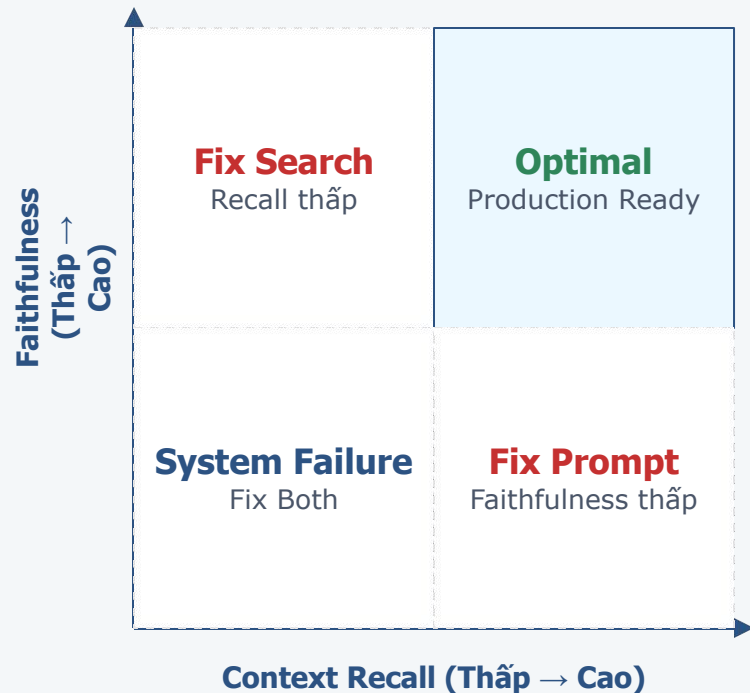
## So Sánh Hiệu Suất Context Recall





## Nhìn vào điểm số để bắt bệnh

- **Recall Cao + Faithfulness Thấp:**  
Tìm đúng tài liệu, nhưng model bị ảo giác hoặc bị lú vì prompt viết quá dở.  
→ **Sửa Generation.**
- **Recall Thấp + Faithfulness Cao:** Hệ thống đang ngoan ngoãn nói "Tôi không biết" vì không tìm thấy tài liệu.  
→ **Sửa Indexing/Retrieval.**



## Phân tích kỹ thuật & kinh tế

- Kỹ thuật Cross-encoder Reranker giúp tăng Answer Relevance thêm **5%**.
- Nhưng nó làm thời gian phản hồi (Latency) tăng từ **1s lên 4s**, và chi phí Server **tăng gấp đôi**.
- *Bài toán của Kỹ sư trưởng: 5% độ chính xác đó có đáng giá với trải nghiệm chậm chạp của người dùng không?*

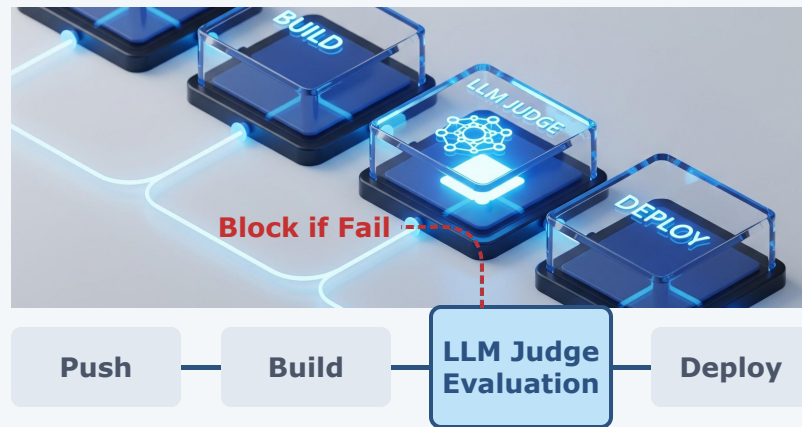


**5% Quality  
Boost**

**\$500/month  
extra**  
+ 3s latency delay

## Kiểm soát hành vi AI trong Pipeline

- Code RAG không giống code Web. Khi đẩy code RAG lên Production, bạn không test hàm/logic, bạn test "**Hành vi của AI**".
- Hãy tích hợp vòng lặp RAGAS vào GitHub Actions. Nếu điểm số **Faithfulness < 80%**, hệ thống tự động **block lệnh Deploy**.



## 4.3 Preview: The Agentic Future

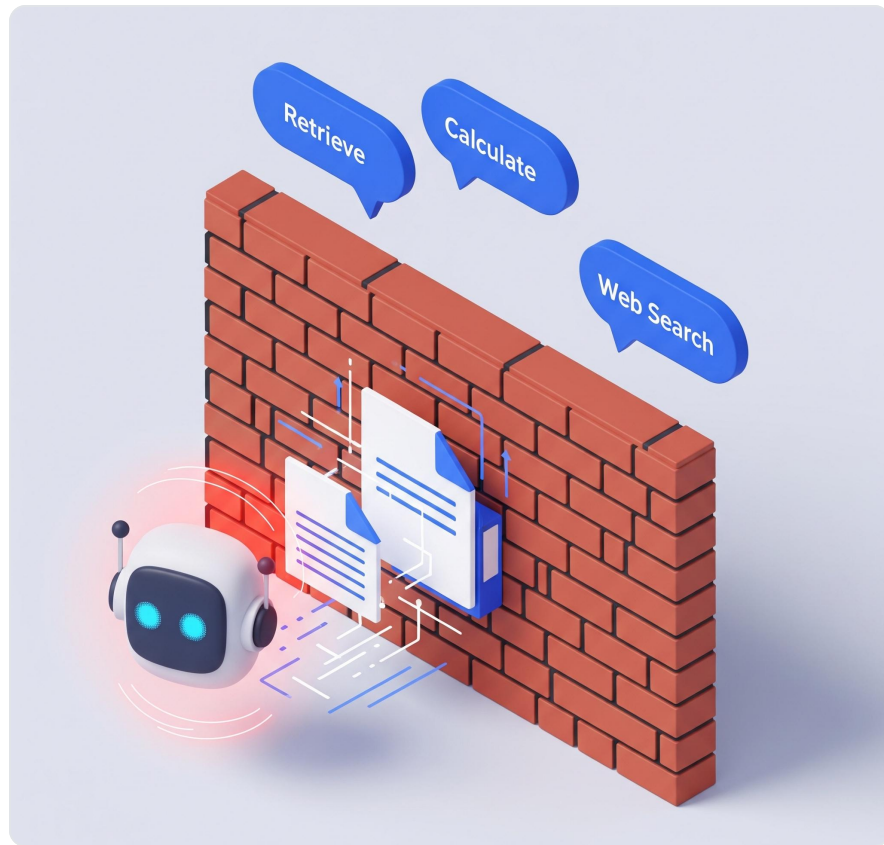
---

Prepare for the agentic future where the LLM evolves into a reasoning engine, and your complex retrieval pipeline becomes just one tool in a multi-agent workflow.

**RAG truyền thống là luồng một chiều:** Nhận câu hỏi → Tìm 1 lần → Trả lời.

*"Dựa vào báo cáo tài chính quý 1, hãy lấy doanh thu trừ đi chi phí nhân sự và so sánh tỷ lệ đó với đối thủ Apple"*

**Hạn chế:** RAG không biết làm toán phức tạp và không thể tự động tìm kiếm thông tin bên ngoài.

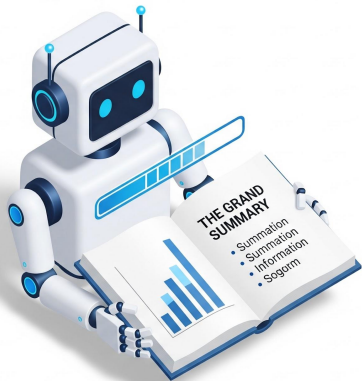


# Chuyển Đổi Mô Hình: Từ RAG Sang Agent

## RAG

### LLM là "Cái miệng"

Tổng hợp thông tin đã được mớm sẵn từ hệ thống truy xuất.



## AGENT

### LLM là "Bộ não"

Reasoning Engine: Tự lập kế hoạch, quyết định công cụ và thực hiện vòng lặp (Loop) xử lý.



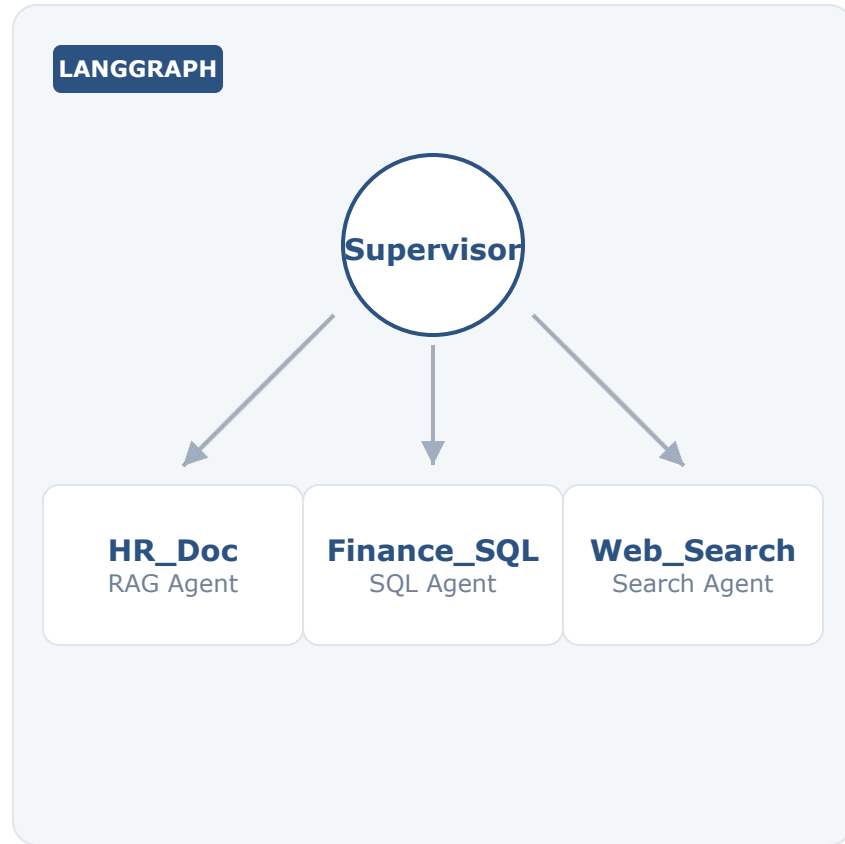
# Retriever Giờ Chỉ Là Một "Công Cụ"

- Trong thế giới Agent, toàn bộ module Retrieval khổng lồ ta vừa học hôm nay sẽ được đóng gói lại thành một hàm Python đơn giản:  
**`search_internal_docs(query: str)`**.
- LLM sẽ tự quyết định: "À, câu hỏi này cần luật nội bộ, mình sẽ gọi Tool này. Câu hỏi kia hỏi về thời tiết, mình sẽ không gọi Tool này."



# Multi-Agent Systems

- Khi hệ thống lớn lên, một Agent không thể ôm đồm mọi việc (quá tải System Prompt).
- Cần chia nhỏ thành các Worker (Nhân sự): 1 RAG Agent chuyên đọc tài liệu, 1 SQL Agent chuyên đọc số liệu, 1 Supervisor Agent làm sắp chỉ việc.
- **Ngày 09:** Chúng ta sẽ dùng LangGraph để vẽ sơ đồ giao tiếp cho các Agent này.





# Hands-on 8

Biến artifact Day 07 thành full RAG pipeline có retrieval, prompt grounding, test set.

5

Nâng cấp hệ thống Day 07 để trả lời grounded hơn, đo được hơn, và dễ giải thích hơn với stakeholder kỹ thuật lẫn sản phẩm.

1. Index bộ tài liệu domain nhỏ với metadata rõ ràng
2. Build baseline retrieval + answer function
3. Thử hybrid hoặc rerank ở mức tối thiểu nếu phù hợp
4. Tạo 10 test questions với expected evidence
5. Chấm kết quả theo scorecard trước và sau tuning

## Code + data

- script indexing
- retrieval / answer function
- bộ docs nhỏ đã index

## Eval artifact

- 10 test questions
- expected answer / evidence
- scorecard trước và sau tuning

**Lưu ý:** Không cần build hệ thống phức tạp. Điều quan trọng là chứng minh được **vì sao bản tuning tốt hơn baseline**.

1. **Index sạch chưa?** text, metadata, freshness ổn chưa?
2. **Retrieve đúng chưa?** dense-only có đang miss keyword hay alias không?
3. **Có cần rerank không?** top-k hiện tại có trùng lặp nhiều không?
4. **Prompt có grounded không?** model có biết từ chối khi thiếu chứng cứ không?
5. **Eval có nói thật không?** testset đã đủ các câu khó và câu mơ hồ chưa?

## Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 RAG là sự phối hợp giữa **indexing**, **retrieval**, và **generation**; thiếu bước nào cũng dễ làm hệ thống trả lời sai.
- 2 **Retrieval quality > generation polish** trong nhiều bài toán thực tế. Search sai thì prompt đẹp đến đâu cũng khó cứu.
- 3 **Hybrid retrieval và rerank** là hai đòn bẩy rất thực dụng khi dense-only bắt đầu bộc lộ giới hạn.
- 4 RAG muốn tốt lên phải có **test set + scorecard + A/B tuning**, không thể dựa vào cảm giác.

1. Lewis et al. (2020), *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.
2. OpenAI Docs, *Retrieval Guide* và *File Search Guide*.
3. LangChain, *RAG from Scratch* notebooks.
4. LlamaIndex Docs, *Starter Example*.
5. RAGAS Docs, *Evaluation metrics for RAG systems*.
6. Cohere Docs, *Rerank overview*.

# Hỏi & Đáp

*Bạn đang thiếu model mạnh hơn, hay đang thiếu một pipeline retrieval và evaluation đủ kỷ luật?*